Natural Language Proofs for the Formalized Riemannian Geometry Package

Shengrong Wu

March 31, 2026

Contents

Introduction **4**

1 Riemannian Metric **5**
1.1 Main results . . . 5
1.2 Definitions . . . 5
1.3 Key lemmas . . . 5
1.4 Smooth metric (`Metric/SmoothMetric.lean`) . . . 6

2 Levi-Civita Connection **7**
2.1 Main theorem . . . 7
2.2 Proof architecture . . . 7
2.2.1 Abstract framework (`LeviCivita/Basic.lean`) . . . 7
2.2.2 Koszul formula and uniqueness (`LeviCivita/Koszul.lean`) . . . 7
2.2.3 Coordinate uniqueness (`LeviCivita/LocalExistence.lean`) . . . 8
2.2.4 Extensionality from the inverse pair (`LeviCivita/LocalExistence.lean`) . . . 8

3 ODE Infrastructure **9**
3.1 Main results . . . 9
3.2 Proof architecture . . . 9
3.2.1 Flow Lipschitz estimates (`ODE/Core.lean`) . . . 9
3.2.2 Linearised solution operator (`ODE/Linearized.lean`) . . . 9
3.2.3 Gronwall bound on the linearisation error (`ODE/FlowFDeriv.lean`) . . . 9

4 Parallel Transport **11**
4.1 Main theorem . . . 11
4.2 Proof architecture . . . 11
4.2.1 Fields along a curve and the transport ODE (`ParallelTransport/Basic.lean`) . . . 11
4.2.2 Isometry property (`ParallelTransport/Isometry.lean`) . . . 11
4.2.3 Parallel frames (`ParallelTransport/Frames.lean`) . . . 11

5 Curvature Tensor **12**
5.1 Main results . . . 12
5.2 Proof architecture . . . 12
5.2.1 Curvature operator and tensor (`Curvature/Basic.lean`) . . . 12
5.2.2 First-pair skew-symmetry (`Curvature/Symmetries.lean`) . . . 12
5.2.3 First Bianchi identity (`Curvature/Symmetries.lean`) . . . 12
5.2.4 Sectional and Ricci curvature (`Curvature/SectionalRicci.lean`) . . . 12

6 Geodesic ODE **13**
6.1 Main theorem . . . 13
6.2 Proof architecture . . . 13
6.2.1 State space and geodesic spray (`Geodesic/Basic.lean`, `Geodesic/Spray.lean`) . . . 13
6.2.2 Acceleration bound (`Exponential/DifferentialAtZero.lean`) . . . 13
6.2.3 Local existence (`Geodesic/LocalExistence.lean`) . . . 13
6.2.4 Local uniqueness (`Geodesic/LocalUniqueness.lean`) . . . 13

7 Exponential Map **14**
7.1 Main theorems . . . 14
7.2 Proof architecture . . . 14
7.2.1 Coordinate exponential map (`Exponential/Basic.lean`) . . . 14
7.2.2 Differential at zero (`Exponential/DifferentialAtZero.lean`) . . . 14
7.2.3 Local inverse via the Inverse Function Theorem (`Exponential/LocalInverse.lean`) . . . 14
7.2.4 Radial source (`Exponential/RadialBridge.lean`) . . . 15
7.2.5 Gauss lemma at the origin (`Exponential/GaussLemma.lean`) . . . 15

8 Local Minimization: Normal Neighborhoods **16**
8.1 Normal neighborhoods (`Minimization/NormalNeighborhoods.lean`) . . . 16

9 Riemannian Volume and Polar Coordinates **17**
9.1 Main result . . . 17
9.2 Proof architecture . . . 17
9.2.1 Riemannian volume (`Measure/RiemannianVolume.lean`) . . . 17
9.2.2 Polar decomposition (`Measure/PolarCoordinates.lean`) . . . 17
9.2.3 Pullback density (`Measure/PullbackDensity.lean`) . . . 17
9.2.4 Exponential map Jacobian (`Measure/ExpJacobian.lean`) . . . 17

10 Variation Formulas **18**
10.1 Main theorem . . . 18
10.2 Proof architecture . . . 18
10.3 Abstract one-parameter connection calculus (`Variation/Basic.lean`, `Variation/Variation.lean`) . . . 18
10.4 Component bridge and coordinate curvature variation (`Variation/ComponentBridge.lean`, `Variation/CoordinateVariatio`
10.5 Levi-Civita bridge (`Variation/LeviCivitaVariationBridge.lean`) . . . 19
10.6 Ricci and scalar contractions (`Variation/RicciScalarVariation.lean`) . . . 19

11 Jacobi Fields **20**
11.1 Main theorem . . . 20
11.2 Proof architecture . . . 20
11.2.1 Abstract Jacobi field predicate (`Jacobi/Basic.lean`) . . . 20
11.2.2 First-order system reduction (`Jacobi/ExistenceUniqueness.lean`) . . . 20

12 Second Bianchi Identity **21**
12.1 Main theorem . . . 21
12.2 Proof architecture . . . 21
12.3 Abstract local connection algebra (`SecondBianchi/Basic.lean`) . . . 21
12.4 Abstract second Bianchi identity (`SecondBianchi/SecondBianchi.lean`) . . . 21
12.5 Component bridge (`SecondBianchi/ComponentBridge.lean`) . . . 22
12.6 Coordinate instantiation (`CoordinateBianchi.lean`, `ClassicalCoordinateBianchi.lean`, `ConcreteCoordinateBianchi.le`
12.7 Smooth instantiation (`SmoothCoordinateScaffold.lean`, `SmoothCoordinateBianchi.lean`) . . . 22

13 Bochner Identity **23**
13.1 Main theorem . . . 23
13.2 Proof architecture . . . 23
13.3 Bundle-valued connection algebra (`Bochner/Basic.lean`) . . . 23
13.4 Abstract curvature-action theorem (`Bochner/Bochner.lean`) . . . 24
13.5 Component bridge and coordinate commutator (`Bochner/ComponentBridge.lean`, `Bochner/CoordinateBochner.lean`) 24
13.6 Trace step and scalar identity (`Bochner/ClassicalCoordinateBochner.lean`, `Bochner/ConcreteCoordinateBochner.lean`)

14 Weitzenböck Identity **25**
14.1 Main theorem . . . 25
14.2 Proof architecture . . . 25
14.3 Abstract one-form Hodge algebra (`Weitzenbock/Basic.lean`, `Weitzenbock/Weitzenbock.lean`) . . . 25
14.4 Coordinate package and Bochner trace bridge (`CoordinateWeitzenbock.lean`, `ClassicalCoordinateWeitzenbock.lean`) 25
14.5 Concrete formula and pairing corollary (`ConcreteCoordinateWeitzenbock.lean`) . . . 26

15 Comparison Geometry **27**
15.1 Model space ODE solutions (`Comparison/ODEModels.lean`) . . . 27
15.2 Scalar Sturm comparison theorem (`Comparison/SturmComparison.lean`) . . . 27
15.2.1 Wronskian and its derivative . . . 27
15.2.2 Wronskian is antitone . . . 28
15.2.3 Initial value of the Wronskian . . . 28
15.2.4 Comparison ratio is antitone and bounded above by 1 . . . 28

15.3 Scalar Jacobi comparison bridge (`Comparison/ScalarJacobiComparison.lean`) . . . 28
15.4 Rauch norm core (`Comparison/RauchNormCore.lean`) . . . 28
15.5 Scalar convexity for Cartan–Hadamard (`Comparison/ScalarConvexity.lean`) . . . 29
15.6 Bishop–Gromov monotonicity (`Comparison/BishopGromovMonotonicity.lean`) . . . 29

16 Hopf–Rinow: Metric Foundations **30**
16.1 Properness and completeness (`HopfRinow/Properness.lean`) . . . 30
16.2 Distance realizers and minimizing geodesics (`HopfRinow/DistanceRealizer.lean`) . . . 30
16.3 Length-compactness via Arzelà–Ascoli (`HopfRinow/LengthCompactness.lean`) . . . 31
16.4 Local compactness from normal neighborhoods (`HopfRinow/LocalCompactness.lean`) . . . 31

17 Summary of the Consolidated Document **32**

Introduction

This document presents the natural-language proof companion to a Lean 4 / Mathlib formalization of Riemannian geometry, developed as an AI-assisted proof-engineering project. The goal of the library is a rigorous, machine-checkable account of Riemannian geometry from its metric-theoretic foundations through geodesic theory, curvature, variational calculus, and global comparison geometry.

Architecture

The library deliberately maintains two complementary presentation styles throughout. The *abstract / type-class layer* states theorems over abstract `AddGroup`, `SMul`, and custom `ConnectionContext` types, yielding results that are coordinate-independent and directly reusable across Mathlib's differential-geometry infrastructure. The *coordinate / `Fin n` layer* restates the same results in explicit $\mathbb{R}^n$-component form, enabling direct connection to Mathlib's ODE and measure-theory libraries. Both layers coexist for most areas of the library and are linked by bridge lemmas that translate between them.

Completed formalization

The following modules have been formalized and verified in Lean 4. The full project builds cleanly ($\approx$ 2949 Lake jobs, verified 2026-03-31) with only two localized `sorry` placeholders in active-development leaf files.

Metric infrastructure. The Metric module provides the foundational Riemannian metric theory at both layers. At the abstract level it defines `tangentInner` and proves symmetry, Cauchy–Schwarz, positivity, and the $\|v\|^2$ identity. At the coordinate level it defines positive-definite metric tensors g_{ij}, their inverses g^{ij}, the musical isomorphisms (index raising and lowering), and the smooth Christoffel-compatible field type `SmoothTensor2Field`.

Levi-Civita connection. The LeviCivita module proves existence and uniqueness of the Levi-Civita connection. At the abstract level, metric compatibility (`MetricCompatible`), torsion-freeness (`TorsionFree`), and the combined `IsLeviCivita` predicate are defined, and uniqueness is derived from the Koszul formula. At the coordinate level the Christoffel symbols $\Gamma^k_{ij} = \frac{1}{2} g^{kl}(\partial_i g_{jl} + \partial_j g_{il} - \partial_l g_{ij})$ are given explicitly, with global existence verified by `Globalization.lean`.

ODE infrastructure. The ODE module supplies the compact autonomous ODE layer used throughout the geodesic and exponential-map pipeline. It provides: fixed-time Lipschitz dependence, a uniform first-order Taylor remainder estimate on compact tubes, a short-time linearized equation and its evaluation operator, global extension by overlap uniqueness and piecewise assembly, Fréchet differentiability of a C^1 autonomous flow (`hasFlowFDerivAtFixedTime_of_contDiff`), and fixed-time C^k regularity (`contDiffAt_flow_of_contDiff`). The entire ODE pipeline is clean.

Geodesics. The Geodesic module encodes the geodesic ODE as a first-order system in phase space $\mathrm{Position}_n \times \mathrm{Velocity}_n$. It defines the geodesic acceleration $\ddot{x}^i = -\Gamma^i_{jk}\dot{x}^j\dot{x}^k$, proves local existence via Picard–Lindelöf, local uniqueness from prescribed initial conditions, and Fréchet differentiability of the geodesic flow in the initial state (`hasFlowFDerivAtInitialState_of_smoothChristoffel`). The variational corollary `flow_fderiv_at_initial_of_smoothChristoffel` — giving the derivative of the flow along radial initial-velocity lines — is exported for use by the Gauss lemma.

Parallel transport. The ParallelTransport module defines parallel transport as the solution of a first-order linear ODE along a curve in a trivialized fiber $\mathbb{R}^n$. It proves that the transport operator is a linear isometry under a metric-compatible connection, and that entire orthonormal frames are transported to orthonormal frames.

Curvature. The Curvature module defines the Riemann curvature tensor $R(X,Y)Z = \nabla_X\nabla_Y Z - \nabla_Y\nabla_X Z - \nabla_{[X,Y]}Z$, sectional curvature, and Ricci curvature Ric at the abstract level. Algebraic symmetries — first-pair skew-symmetry, the first Bianchi identity, and the pair-swap symmetry — are all proved.

Algebraic identities: Second Bianchi, Bochner, and Weitzenböck. Three modules providing fully proved algebraic identities require no interface boundaries. SecondBianchi establishes the second Bianchi identity $d_\nabla \Omega = 0$ at the abstract level and derives the classical cyclic sum $\nabla R = 0$ in coordinates via bridge lemmas. Bochner proves the Bochner formula $\nabla^*\nabla = \Delta_H - \mathcal{R}$ with both abstract and concrete coordinate versions. Weitzenbock proves the Weitzenböck identity $\Delta_H\,\alpha = \nabla^*\nabla\,\alpha + \mathcal{R}\,\alpha$ for one-forms at all three levels (abstract, coordinate, and concrete).

Exponential map. The Exponential module defines the local exponential map $\exp_p\colon T_pM \to M$ in coordinates (`coordinateExp`), proves its differentiability (`coordinateExpHasFDerivAtOnSource`), establishes that the Fréchet derivative at the origin is the identity (`fderiv(coordinateExp p) 0 = id`), and proves the Gauss lemma in both the algebraic and radial-pairing forms. Normal coordinates are constructed via the inverse function theorem (LocalInverse).

Minimization and normal neighborhoods. The Minimization module proves that normal neighborhoods are open (`normalNeighbo` that short geodesics are length-minimizing (`shortGeodesicsMinimize_of_input`), and establishes the radial comparison between metric distance and geodesic length (`radialComparison_metric_of_input`).

Measure and Riemannian volume. The Measure module defines the Riemannian volume form via the metric tensor (`densityOfTen` the pullback measure under the exponential map (`pullbackMeasure`), the Jacobian bound of the exponential map (`ExpJacobian`) using the Hadamard determinant inequality, and polar coordinate decomposition (`HasPolarDecomposition`).

Variational calculus. The Variation module develops curve energy and length functionals, first and second variation formulas (`firstVariationOfEnergy`, `secondVariationOfEnergy_eq_indexForm`), the energy-criticality characterization of geodesics, and the Ricci and scalar curvature variation formulas.

Jacobi fields. The Jacobi module defines Jacobi fields as solutions of $\nabla_T \nabla_T J + R(J,T)T = 0$, proves existence and uniqueness from prescribed initial data via Picard–Lindelöf and Gronwall estimates, defines conjugate points, and constructs the bridge between geodesic variations and Jacobi fields (`jacobiField_of_geodesicVariation`).

Comparison geometry. The Comparison module contains the Sturm comparison theorem (`scalarComparison_of_subsolution`), the Rauch comparison theorem (`dexp_normComparison_of_rauch`), local volume comparison from Rauch bounds, Bishop–Gromov monotonicity, the Cartan–Hadamard theorem, Myers's theorem, and the Bishop–Gromov volume comparison inequality.

Hopf–Rinow theorem. The HopfRinow module proves local compactness of Riemannian manifolds, the equivalence of metric and geodesic completeness, existence of minimizing geodesics in complete manifolds, and the Arzela–Ascoli-based length-compactness lemma, culminating in the Hopf–Rinow theorem.

1 Riemannian Metric

1.1 Main results

Main Theorem 1.1 (Coordinate metric infrastructure)**.** *Let $g : \mathrm{Fin}\, n \to \mathrm{Fin}\, n \to \mathbb{R}$ be a symmetric positive-definite coordinate metric tensor, and let g^{-1} be its inverse ($\sum_k g^{-1}_{ik} g_{kj} = \delta_{ij}$). The associated bilinear form*

$$B_g(v,w) := \sum_{i,j} v_i\, g_{ij}\, w_j$$

is symmetric, and the pair (g, g^{-1}) is uniquely determined by either g or g^{-1}. Moreover, the bilinear form is extensional*: $B_g(u,Z) = B_g(v,Z)$ for all Z implies $u = v$.*

1.2 Definitions

Definition 1.1 (Coordinate metric)**.** *A* coordinate metric *is a term of type* `Tensor2`$(n) = \mathrm{Fin}\, n \to \mathrm{Fin}\, n \to \mathbb{R}$. *The associated structures are:*

- ***Quadratic form***: `quadraticForm`$(g,v) = \sum_{i,j} v_i\, g_{ij}\, v_j$.
- ***Bilinear form***: `bilinForm`$(g,v,w) = \sum_{i,j} v_i\, g_{ij}\, w_j$.
- ***IsSymmetric***: $\forall i\, j,\ g_{ij} = g_{ji}$.
- ***IsPositiveDefinite***: `IsSymmetric`(g) *and* $\forall v \neq 0,$ `quadraticForm`$(g,v) > 0$.
- ***IsInversePair***(g, g^{-1}): $\forall i\, j,\ \sum_k g^{-1}_{ik}\, g_{kj} = \delta_{ij}$.
- ***basisVector***(i): *the function* $e_i = \mathrm{Pi.single}(i,1)$.

1.3 Key lemmas

Lemma 1.2 (Symmetry of the bilinear form)**.** *If g is symmetric, then $B_g(v,w) = B_g(w,v)$ for all v, w.*

Proof. `bilinForm_comm`: by `Finset.sum_comm`, interchange the order of summation to get $\sum_i \sum_j v_i g_{ij} w_j$. Relabelling $i \leftrightarrow j$ and using symmetry $g_{ji} = g_{ij}$ (hypothesis `hg`), followed by `ring`, gives $\sum_j \sum_i w_j g_{ji} v_i = \sum_j \sum_i w_j g_{ij} v_i = B_g(w,v)$. □

Lemma 1.3 (Basis evaluation)**.** $B_g(e_i, e_j) = g_{ij}$.

Proof. `bilinForm_basis`: the outer sum $\sum_a$ over $\mathrm{Fin}\, n$ is concentrated at $a = i$ because $e_i(a) = \mathrm{Pi.single}(i,1)(a) = 0$ for $a \neq i$. `Fintype.sum_eq_single` reduces to $a = i$. The inner sum $\sum_b$ is concentrated at $b = j$ by the same argument for e_j. The result is $1 \cdot g_{ij} \cdot 1 = g_{ij}$. □

Lemma 1.4 (Inverse-pair diagonal and off-diagonal identities)**.** *If (g, g^{-1}) is an inverse pair, then for all i, j,*

$$\sum_k g^{-1}_{ik}\, g_{kj} = \begin{cases} 1 & i = j, \\ 0 & i \neq j. \end{cases}$$

Proof. `inversePair_apply` is the definition; `inversePair_diag` instantiates at $i = j$ and uses `simpa`; `inversePair_offDiag` uses `simp` with $i \neq j$ to simplify the `if`-expression to 0. □

Lemma 1.5 (Nondegenerate quadratic form)**.** *If g is positive definite and $B_g(v, Z) = 0$ for all Z, then $v = 0$.*

Proof. `IsPositiveDefinite.nondegenerate`: take $Z = v$. Then $B_g(v, v) = \texttt{quadraticForm}(g, v) = 0$. Positive definiteness implies $v = 0$. □

1.4 Smooth metric (`Metric/SmoothMetric.lean`)

Definition 1.6 (Smooth metric)**.** *A `SmoothMetric` is a Mathlib `RiemannianBundle` metric satisfying the smooth-manifold version of symmetry and positive definiteness. `ContMDiffMetric` packages the same data with a finite smoothness order.*

Lemma 1.7 (`smoothMetric_inner_symm`)**.** *For a `SmoothMetric`, the induced inner product is symmetric: $g_p(u, v) = g_p(v, u)$ for all p and all tangent vectors u, v at p.*

Proof. Follows directly from the abstract `Inner.comm` instance for Mathlib's `InnerProductSpace`, which is part of the `RiemannianBundl` typeclass. □

2 Levi-Civita Connection

2.1 Main theorem

Main Theorem 2.1 (Existence and uniqueness of the Levi-Civita connection)**.** *Let g be a symmetric metric pairing on a vector space V that is extensional (non-degenerate in the first slot), and let C be a connection context on V. There is at most one covariant derivative ∇ satisfying simultaneously metric compatibility and torsion-freeness. In coordinates, the unique solution is given by the Christoffel symbols*

$$\Gamma^k{}_{ij} = \tfrac{1}{2}\, g^{kl}\big(\partial_i g_{jl} + \partial_j g_{il} - \partial_l g_{ij}\big).$$

2.2 Proof architecture

2.2.1 Abstract framework (`LeviCivita/Basic.lean`)

A *connection context* (V, C) consists of a Lie bracket $[\,\cdot\,,\cdot\,] : V \times V \to V$ and a scalar derivation $\partial : V \to S$ satisfying Leibniz over a scalar type S. The abstract predicates are:

- `CovariantDerivative`: a map $\nabla : V \times V \to V$ satisfying the Leibniz rule $\nabla_X(fY) = (\partial_X f)Y + f\nabla_X Y$.
- `MetricCompatible`: for all X, Y, Z, $\partial_X g(Y, Z) = g(\nabla_X Y, Z) + g(Y, \nabla_X Z)$.
- `TorsionFree`: for all X, Y, $\nabla_X Y - \nabla_Y X = [X, Y]$.
- `MetricExtensional`: if $g(U, Z) = g(V, Z)$ for all Z, then $U = V$.

2.2.2 Koszul formula and uniqueness (`LeviCivita/Koszul.lean`)

Definition 2.1 (Koszul right-hand side)**.** *For a connection context C, a metric pairing g, and vectors X, Y, Z, define*

$$\mathrm{koszulRHS}(X, Y, Z) := \partial_X g(Y, Z) + \partial_Y g(Z, X) - \partial_Z g(X, Y) + g([X, Y], Z) - g([Y, Z], X) + g([Z, X], Y).$$

Definition 2.2 (Koszul formula)**.** *A covariant derivative ∇ satisfies the* Koszul formula *if*

$$2 \cdot g(\nabla_X Y, Z) = \mathrm{koszulRHS}(X, Y, Z) \quad \textit{for all } X, Y, Z.$$

Theorem 2.3 (Uniqueness from Koszul formula)**.** *If ∇ and ∇' both satisfy the Koszul formula with respect to an extensional metric g, then $\nabla = \nabla'$.*

Proof. Since both satisfy the Koszul formula,

$$2 \cdot g(\nabla_X Y, Z) = \mathrm{koszulRHS}(X, Y, Z) = 2 \cdot g(\nabla'_X Y, Z)$$

for all X, Y, Z. The scalar 2 is cancellable (hypothesis `hcancel`), giving $g(\nabla_X Y, Z) = g(\nabla'_X Y, Z)$ for all Z. Extensionality then forces $\nabla_X Y = \nabla'_X Y$ for all X, Y. Lean proves this by `funext` on the underlying function and `subst` after establishing function equality. □

Theorem 2.4 (Koszul formula follows from metric compatibility and torsion-freeness)**.** *If g is symmetric ($g(X, Y) = g(Y, X)$), subtractive in the first slot, and ∇ is a Levi-Civita connection (metric compatible and torsion-free), then ∇ satisfies the Koszul formula.*

Proof. Write $[X, Y] = \nabla_X Y - \nabla_Y X$ (torsion-freeness), and similarly for $[Y, Z]$ and $[Z, X]$. This substitution rewrites the three bracket terms in koszulRHS as covariant derivative differences. Then expand metric compatibility three times:

$$\partial_X g(Y, Z) = g(\nabla_X Y, Z) + g(Y, \nabla_X Z),$$

and symmetrically for cyclic permutations of X, Y, Z. Using metric symmetry ($g(Y, \nabla_X Z) = g(\nabla_X Z, Y)$, etc.), six terms appear in the right-hand side. Four cancel in pairs, leaving

$$g(\nabla_X Y, Z) + g(\nabla_X Y, Z) = 2 \cdot g(\nabla_X Y, Z).$$

In Lean, after the substitutions the identity reduces to a linear arithmetic statement and is closed by `linarith` with the six metric-compatibility and symmetry hypotheses. □

2.2.3 Coordinate uniqueness (`LeviCivita/LocalExistence.lean`)

At the coordinate level, uniqueness takes the following form.

Definition 2.5 (Metric contraction identity)**.** *A Christoffel symbol* ${\Gamma^k}_{ij}$ satisfies the metric contraction *if for every* m, i, j,

$$\sum_k g_{mk}\, {\Gamma^k}_{ij} = \tfrac{1}{2}\big(\partial_i g_{jm} + \partial_j g_{im} - \partial_m g_{ij}\big).$$

Theorem 2.6 (Coordinate uniqueness by inverse recovery)**.** *If* g *and* g^{-1} *form an inverse pair* ($\sum_k g^{-1}_{pm} g_{mk} = \delta_{pk}$)*, and* Γ, Γ' *both satisfy the metric contraction identity, then* $\Gamma = \Gamma'$.

Proof. For any solution Γ of the metric contraction, multiply both sides by g^{-1}_{pm} and sum over m. The calculation runs:

$$\begin{aligned}
{\Gamma^p}_{ij} &= \sum_k \delta_{pk}\, {\Gamma^k}_{ij} = \sum_k \Big(\sum_m g^{-1}_{pm}\, g_{mk}\Big) {\Gamma^k}_{ij} \\
&= \sum_m g^{-1}_{pm} \Big(\sum_k g_{mk}\, {\Gamma^k}_{ij}\Big) = \sum_m g^{-1}_{pm} \cdot \tfrac{1}{2}(\partial_i g_{jm} + \partial_j g_{im} - \partial_m g_{ij}).
\end{aligned}$$

The interchange of summations uses Finset commutativity (`Finset.sum_comm`) and associativity (`Finset.mul_sum`). Since the right-hand side depends only on g and ∂g, not on Γ, any two solutions must agree. Lean closes with `funext` after establishing the recovery formula for each. □

The local existence and uniqueness theorem follows immediately:

Theorem 2.7 (Coordinate existence and uniqueness)**.** *For symmetric positive-definite* g *with inverse* g^{-1}*, there exists a unique* Γ *satisfying the metric contraction identity, namely* ${\Gamma^k}_{ij} = \sum_m g^{-1}_{km} \cdot \frac{1}{2}(\partial_i g_{jm} + \partial_j g_{im} - \partial_m g_{ij})$.

Proof. Existence: `localLeviCivitaConnection_satisfiesMetricContraction` verifies the formula by applying `metric_contract_le`
Uniqueness: Theorem 2.6. The Lean proof is a single `refine ⟨..., ..., ...⟩` on the `ExistsUnique` goal. □

2.2.4 Extensionality from the inverse pair (`LeviCivita/LocalExistence.lean`)

Theorem 2.8 (Inverse pair implies extensionality)**.** *If* g *and* g^{-1} *satisfy the inverse-pair identity, and* g *is symmetric, then the coordinate bilinear form* $B_g(U, V) = \sum_{ij} U_i g_{ij} V_j$ *is extensional.*

Proof. Suppose $B_g(U, Z) = B_g(V, Z)$ for all Z. Then $B_g(U - V, Z) = 0$ for all Z. Testing $Z = e_m$ (the m-th basis vector), symmetry of g gives $\sum_j (U - V)_j g_{jm} = 0$ for all m. Multiplying by g^{-1}_{pm} and summing over m:

$$(U - V)_p = \sum_k \delta_{pk} (U - V)_k = \sum_m g^{-1}_{pm} \sum_j g_{mj} (U - V)_j = 0.$$

The same Finset.sum_comm exchange as in Theorem 2.6 handles the sum interchange; the final step uses $B_g(U - V, e_m) = 0$. □

3 ODE Infrastructure

3.1 Main results

Main Theorem 3.1 (Fixed-time Fréchet differentiability for C^1 flows)**.** *Let $F : E \to E$ be C^1 on a finite-dimensional normed space E. Let $\Phi : E \times \mathbb{R} \to E$ be a local flow satisfying $\frac{d}{dt}\Phi(z,t) = F(\Phi(z,t))$ on a compact time interval $[t_0, t_0 + \varepsilon]$, remaining in a ball around z_0. Then for each fixed t, the map $z \mapsto \Phi(z,t)$ is Fréchet differentiable, with derivative equal to the time-t evaluation of the linearised equation*

$$J'(s) = (DF)_{\Phi(z,s)}\, J(s), \qquad J(t_0) = h.$$

3.2 Proof architecture

3.2.1 Flow Lipschitz estimates (`ODE/Core.lean`)

The core module provides the fundamental fixed-time Lipschitz estimate.

Theorem 3.1 (Fixed-time flow Lipschitz)**.** *If F is Lipschitz with constant K on a ball, and two trajectories $\Phi(z_1, \cdot)$ and $\Phi(z_2, \cdot)$ both remain in the ball on $[t_0 - \varepsilon, t_0 + \varepsilon]$, then for all t in that interval,*

$$\|\Phi(z_1,t) - \Phi(z_2,t)\| \le \|z_1 - z_2\| \cdot e^{K\varepsilon}.$$

Proof. The Lean theorem `flow_fixedTime_dist_le` bounds the distance between two trajectories via the Gronwall inequality (`Mathlib.Analysis.ODE.Gronwall`). The difference $d(t) = \|\Phi(z_1,t) - \Phi(z_2,t)\|$ satisfies $d'(t) \le Kd(t)$ by the Lipschitz condition on F, with $d(t_0) = \|z_1 - z_2\|$. Gronwall's inequality then gives the exponential bound. □

3.2.2 Linearised solution operator (`ODE/Linearized.lean`)

Definition 3.2 (Linearised solution data)**.** *A `LinearizedSolutionData` for the operator $A(s) = (DF)_{\Phi(z,s)}$ is a record consisting of:*

- *a curve $J : \mathbb{R} \to E$ (the solution field),*
- *the initial condition $J(t_0) = h$,*
- *the ODE property: $J'(s) = A(s)\, J(s)$ on $[t_0, t_0 + \varepsilon]$,*
- *continuity of J on the time interval.*

Theorem 3.3 (Existence of the linearised solution)**.** *For any initial vector h, the linearised equation has a solution `LinearizedSolution` on $[t_0, t_0 + \varepsilon]$.*

Proof. `exists_linearizedSolutionData` invokes Mathlib's Picard–Lindelöf theorem on the time-dependent linear ODE $J' = A(s)J$. Linearity and continuity of $A(s)$ give the required Lipschitz condition. The resulting solution is linear in h: `linearizedSolution` and `linearizedSolution_smul_eq` prove additivity and homogeneity by the uniqueness part of Picard–Lindelöf. The `linearizedSolut` lemma applies Gronwall to bound $\|J(t)\|$ exponentially in time. □

Theorem 3.4 (Fixed-time evaluation is a continuous linear map)**.** *The map $h \mapsto J_h(t)$ (evaluation of the linearised solution at fixed t) is a continuous linear map $E \to E$.*

Proof. `exists_linearizedSolution_clm` packages the linearity (from `linearizedSolution_add_eq` and `linearizedSolution_smul_` and continuity (from the Gronwall norm bound and continuity of J) into a `ContinuousLinearMap`. □

3.2.3 Gronwall bound on the linearisation error (`ODE/FlowFDeriv.lean`)

Theorem 3.5 (Linearisation error bound)**.** *Under the hypotheses of Theorem 3.1, let J_h be the linearised solution with $J_h(t_0) = h$. The error*

$$e(t) := \Phi(z+h,t) - \Phi(z,t) - J_h(t)$$

satisfies

$$\|e(t)\| \le \mathrm{gronwallBound}(0, M, \varepsilon_{\mathrm{rem}}\|h\|e^{K\varepsilon}, t - t_0)$$

for any $\varepsilon_{\mathrm{rem}} > 0$ for which F has Taylor defect at most $\varepsilon_{\mathrm{rem}}\|y - \Phi(z,s)\|$ along $\Phi(z,\cdot)$.

Proof. The Lean proof `flow_linearisation_error_bound` proceeds in three stages.

Stage 1: $e(t_0) = 0$. By the initial condition $\Phi(z+h,t_0) = z+h$ and $\Phi(z,t_0) = z$ and $J_h(t_0) = h$, we get $e(t_0) = (z+h)-z-h = 0$.

Stage 2: ODE for e. On the interval $[t_0, t_0 + \varepsilon)$,

$$\begin{aligned} e'(s) &= F(\Phi(z+h,s)) - F(\Phi(z,s)) - (DF)_{\Phi(z,s)}\, J_h(s) \\ &= \big[F(\Phi(z+h,s)) - F(\Phi(z,s)) - (DF)_{\Phi(z,s)}(\Phi(z+h,s) - \Phi(z,s))\big] \\ &\quad + (DF)_{\Phi(z,s)}\, e(s). \end{aligned}$$

The bracketed term is the Taylor defect of F, bounded by $\varepsilon_{\mathrm{rem}}\|\Phi(z+h,s)-\Phi(z,s)\| \le \varepsilon_{\mathrm{rem}}\|h\|e^{K\varepsilon}$ using Theorem 3.1. Hence $\|e'(s)\| \le M\|e(s)\| + \varepsilon_{\mathrm{rem}}\|h\|e^{K\varepsilon}$, where $M = \sup\|(DF)_x\|$.

Stage 3: Gronwall. Apply Gronwall's inequality with initial value 0, linear coefficient M, and constant forcing $\varepsilon_{\mathrm{rem}}\|h\|e^{K\varepsilon}$, to conclude the stated bound. In Lean, the derivative of e is assembled from `hasDerivWithinAt_Ici_of_Icc_symmetric` for each trajectory, the linearised solution's derivative property, and a subtraction; `gronwall_bound` provides the explicit Gronwall conclusion. □

Theorem 3.6 (Fixed-time Fréchet differentiability)**.** *The time-t flow map $z \mapsto \Phi(z,t)$ has Fréchet derivative $D_z\Phi(\cdot,t) =$ (fixed-time evaluation of the linearised solution).*

Proof. By Theorem 3.5, with $\varepsilon_{\mathrm{rem}} \to 0$ (using C^1 regularity of F, which guarantees the Taylor defect is $o(\|y-\Phi(z,s)\|)$),

$$\|\Phi(z+h,t) - \Phi(z,t) - J_h(t)\| = o(\|h\|).$$

This is exactly the `HasFDerivAt` little-o characterisation. The Lean proof `hasFlowFDerivAtFixedTime_of_contDiff` packages Theorems 3.5–3.4 into a `HasFDerivAt` instance. □

4 Parallel Transport

4.1 Main theorem

Main Theorem 4.1 (Parallel transport is an isometry)**.** *Let $A : \mathbb{R} \to \mathrm{Mat}(n \times n, \mathbb{R})$ be a continuous family of matrices (the connection along a curve), and let $g : \mathbb{R} \to \mathrm{Mat}(n \times n, \mathbb{R})$ be a continuous family of inner products satisfying the metric-compatibility condition*

$$g'(t) = A(t)^T g(t) + g(t)A(t).$$

For any initial vectors $v, w \in \mathbb{R}^n$, let $V(t), W(t)$ be the parallel transports of v, w along the connection (solutions of $V'(t) = -A(t)V(t)$). Then

$$\langle V(t), W(t)\rangle_{g(t)} = \langle v, w\rangle_{g(0)} \quad \textit{for all } t.$$

4.2 Proof architecture

4.2.1 Fields along a curve and the transport ODE (`ParallelTransport/Basic.lean`)

Definition 4.1 (Along-derivative and parallel field)**.** *A* field along *a parameter interval is a function $V : \mathbb{R} \to \mathbb{R}^n$ (written* `FieldAlong`*). The* along-derivative *is defined by an* `AlongDerivative` *structure specifying the connection matrix $A(t)$; the induced ODE is*

$$V'(t) = \texttt{coordinateParallelRhs}(A, t, V(t)) = \textstyle\sum_j A(t)_{ij} V(t)_j.$$

A field is parallel *(*`IsParallelAlong`*) if it satisfies this ODE at every time.*

Theorem 4.2 (Coordinate parallel transport exists and is unique)**.** *For any initial vector $v_0 \in \mathbb{R}^n$, there exists a unique parallel field V with $V(0) = v_0$. The corresponding* `TransportOperator` *$\mathcal{T}_t : v_0 \mapsto V(t)$ is a well-defined linear map on $\mathbb{R}^n$.*

Proof. `coordinateParallelTransport` is constructed by invoking Mathlib's Picard–Lindelöf theorem on the linear ODE $V'(t) = A(t)V(t)$. Linearity of the right-hand side implies global existence and uniqueness on any compact interval. Linearity of $\mathcal{T}_t$ in v_0 follows from linearity of the ODE and uniqueness: if V_1, V_2 are parallel with $V_i(0) = v_i$, then $V_1 + V_2$ is parallel with initial value $v_1 + v_2$, so $\mathcal{T}_t(v_1 + v_2) = \mathcal{T}_t(v_1) + \mathcal{T}_t(v_2)$. □

4.2.2 Isometry property (`ParallelTransport/Isometry.lean`)

Definition 4.3 (Coordinate pairing)**.** *The* coordinate pairing *at time t is*

$$B_g(t)(v, w) := \texttt{coordinatePairingAt}(g(t), v, w) = \sum_{i,j} v_i\, g(t)_{ij}\, w_j.$$

This coincides with the matrix inner product $v \cdot (g(t)\, w)$, as established by `coordinatePairingAt_eq_dotProduct_mulVec`*.*

Theorem 4.4 (Pairing is constant along parallel transport)**.** *Under the metric-compatibility condition, for parallel fields V, W,*

$$\frac{d}{dt} B_g(t)(V(t), W(t)) = 0.$$

Hence $B_g(t)(V(t), W(t)) = B_g(0)(v, w)$ for all t.

Proof. `hasDerivAt_coordinatePairing` applies the product rule:

$$\frac{d}{dt} B_g(t)(V(t), W(t)) = B_{g'}(t)(V(t), W(t)) + B_g(t)(V'(t), W(t)) + B_g(t)(V(t), W'(t)).$$

Substituting $V'(t) = A(t)V(t)$, $W'(t) = A(t)W(t)$, and $g'(t) = A(t)^T g(t) + g(t)A(t)$ (metric compatibility):

$$\begin{aligned} &B_{g'}(t)(V, W) + B_g(t)(AV, W) + B_g(t)(V, AW) \\ &= \sum_{ij} V_i\big((A^T g + gA)_{ij}\big) W_j + \sum_{ij} (AV)_i g_{ij} W_j + \sum_{ij} V_i g_{ij} (AW)_j = 0. \end{aligned}$$

The cancellation is exact by matrix algebra. Lean establishes this as `coordinatePairingAt_parallel_deriv_eq_zero`.

The conclusion $B_g(t)(V(t), W(t)) = B_g(0)(v, w)$ follows from `eqOn_const_of_zero_deriv`: a differentiable real-valued function with everywhere-zero derivative on a connected interval is constant. □

Corollary 4.5 (`coordinateParallelTransport_preservesPairing`)**.** *The transport operator $\mathcal{T}_t$ satisfies* `PreservesPairing` *and hence* `IsometryAlong`*: for all v, w and all t, $B_g(t)(\mathcal{T}_t v, \mathcal{T}_t w) = B_g(0)(v, w)$.*

4.2.3 Parallel frames (`ParallelTransport/Frames.lean`)

Theorem 4.6 (Transport preserves orthonormal frames)**.** *Let $\{e_i\}_{i=1}^n$ be the standard basis at time 0. The transported family $\{V_i(t) = \mathcal{T}_t e_i\}$ is an orthonormal frame at every t: $B_g(t)(V_i(t), V_j(t)) = \delta_{ij}$.*

Proof. `coordinateParallelFrame_isOrthonormal` applies Theorem 4.4 to each pair (e_i, e_j). At $t = 0$, $B_g(0)(e_i, e_j) = \delta_{ij}$ (the standard basis is orthonormal at time 0). Isometry preservation gives the same value at every time. The `frameOfTransport` structure packages the family of transported fields into a `ParallelFrame`, which is verified to satisfy `IsOrthonormalFrame` at every t. □

5 Curvature Tensor

5.1 Main results

Main Theorem 5.1 (Algebraic symmetries of the curvature tensor)**.** *Let ∇ be any covariant derivative and let C be its connection context. Define the* curvature tensor *by*

$$R(X,Y)Z = \nabla_X \nabla_Y Z - \nabla_Y \nabla_X Z - \nabla_{[X,Y]} Z.$$

Then, as elements of the ambient value group:

1. ***First-pair skew-symmetry**: $R(X,Y)Z = -R(Y,X)Z$.*
2. ***First Bianchi identity**: $R(X,Y)Z + R(Y,Z)X + R(Z,X)Y = 0$.*

5.2 Proof architecture

5.2.1 Curvature operator and tensor (`Curvature/Basic.lean`)

Definition 5.1 (Curvature operator)**.** *Given a `CovariantDerivative` ∇ and a connection context C, the* curvature operator *is*

$$\texttt{curvatureOperator}(\nabla, C)(X,Y)(Z) := \nabla_X(\nabla_Y Z) - \nabla_Y(\nabla_X Z) - \nabla_{[X,Y]} Z.$$

The `CurvatureTensor` structure bundles the curvature as a tri-linear map $R : V \times V \times V \to V$ together with extensionality and congruence lemmas.

5.2.2 First-pair skew-symmetry (`Curvature/Symmetries.lean`)

Theorem 5.2 (First-pair skew-symmetry)**.** *$R(X,Y)Z = -R(Y,X)Z$ for all X, Y, Z.*

Proof. The predicate `FirstPairSkew R` is defined as $\forall X\, Y\, Z,\ R(X,Y)Z = -R(Y,X)Z$. The swap lemma

$$\texttt{firstPairSkew_swap}: \quad R(Y,X)Z = -R(X,Y)Z$$

follows by applying the definition with X and Y exchanged and using `simpa`.

As a consequence, `add_self_eq_zero_of_firstPairSkew` proves $R(X,X)Z + R(X,X)Z = 0$: apply the definition with $X = Y$ to get $R(X,X)Z = -R(X,X)Z$, then use `eq_neg_iff_add_eq_zero.mp` and `simpa`. □

5.2.3 First Bianchi identity (`Curvature/Symmetries.lean`)

Theorem 5.3 (First Bianchi identity)**.** *For all X, Y, Z,*

$$R(X,Y)Z + R(Y,Z)X + R(Z,X)Y = 0.$$

Proof. The predicate `FirstBianchi R` is defined as the cyclic sum being zero. The proof proceeds by establishing two corollaries by direct calculation.

Cyclic rotation (`firstBianchi_rotate`): the expression $R(Y,Z)X + R(Z,X)Y + R(X,Y)Z$ also equals zero. The Lean proof is a single `calc` block:

$$R(Y,Z)X + R(Z,X)Y + R(X,Y)Z = R(X,Y)Z + R(Y,Z)X + R(Z,X)Y = 0,$$

where the rearrangement uses three steps of `add_assoc` and `add_comm`, and the final equality is the original Bianchi hypothesis.

Solving for the third term (`firstBianchi_solve_third`): $R(Z,X)Y = -(R(X,Y)Z + R(Y,Z)X)$. The Lean proof uses `eq_neg_of_add_eq_zero_left` after establishing

$$R(Z,X)Y + (R(X,Y)Z + R(Y,Z)X) = R(X,Y)Z + R(Y,Z)X + R(Z,X)Y = 0$$

via `add_assoc`, `add_comm`, and the Bianchi hypothesis. □

5.2.4 Sectional and Ricci curvature (`Curvature/SectionalRicci.lean`)

Definition 5.4 (Sectional curvature)**.** *For vectors u, v spanning a plane σ, the sectional curvature is*

$$K(\sigma) = K(u,v) = \frac{g(R(u,v)v,\ u)}{g(u,u)g(v,v) - g(u,v)^2}.$$

The Lean definition `SectionalCurvature` stores the unnormalized numerator $g(R(u,v)v, u)$ as a scalar-valued function of the pair (u,v).

Definition 5.5 (Ricci curvature and lower bound)**.** *The Ricci curvature $\mathrm{Ric}(u,v)$ is the trace of $w \mapsto R(w,u)v$ over an orthonormal frame. `RicciCurvature` stores this contraction against a finite family of frame vectors.*

`RicciLowerBound`: a predicate asserting $\mathrm{Ric}(v,v) \geq K\|v\|^2$ for all unit tangent vectors v, encoding the geometric hypothesis used in the Jacobi and comparison modules.

6 Geodesic ODE

6.1 Main theorem

Main Theorem 6.1 (Local existence and uniqueness of geodesics)**.** *Let Γ be a smooth Christoffel field on $\mathbb{R}^n$. For every initial state $(x_0, v_0) \in \mathbb{R}^n \times \mathbb{R}^n$ there exists $\varepsilon > 0$ and a unique curve $\gamma : [-\varepsilon, \varepsilon] \to \mathbb{R}^n \times \mathbb{R}^n$ satisfying the geodesic equation in phase space:*

$$\dot{x}^i = v^i, \qquad \dot{v}^i = -\sum_{j,k} \Gamma^i{}_{jk}(x)\, v^j\, v^k, \qquad (x, v)\big|_{t=0} = (x_0, v_0).$$

6.2 Proof architecture

6.2.1 State space and geodesic spray (`Geodesic/Basic.lean`, `Geodesic/Spray.lean`)

Definition 6.1 (Phase space and spray)**.** *Set $\texttt{Position}(n) = \texttt{Velocity}(n) = \mathbb{R}^n$ and $\texttt{State}(n) = \mathbb{R}^n \times \mathbb{R}^n$. The* geodesic spray *is*

$$\texttt{geodesicSpray}(\Gamma)(x, v) := \big(v,\ \texttt{geodesicAcceleration}(\Gamma)(x, v)\big),$$

where $\texttt{geodesicAcceleration}(\Gamma)(x, v)^i = -\sum_{j,k} \Gamma^i{}_{jk}(x)\, v^j\, v^k$.

$\texttt{IsCoordinateGeodesicOn}(\Gamma, \gamma, I)$ asserts that $\gamma : I \to \texttt{State}(n)$ satisfies the spray ODE: $\gamma'(t) = \texttt{geodesicSpray}(\Gamma)(\gamma(t))$ for all $t \in I$.

6.2.2 Acceleration bound (`Exponential/DifferentialAtZero.lean`)

Lemma 6.2 (Quadratic acceleration bound)**.** *For any smooth Γ and compact ball $\overline{B}(p, R)$, there exists $C \geq 0$ such that for all (x, v) with $x \in \overline{B}(p, R)$,*

$$\|\texttt{geodesicAcceleration}(\Gamma)(x, v)\| \leq C\, \|v\|^2.$$

Proof. For each index triple (i, j, k), compactness of $\overline{B}(p, R)$ and continuity of $x \mapsto \Gamma^i{}_{jk}(x)$ give a bound B_{ijk}. Setting $C = \sum_{i,j,k} B_{ijk}$, a `calc` block bounds each component:

$$\|\texttt{geodesicAcceleration}(\Gamma)(x, v)^i\| = \Big\|\sum_j \sum_k \Gamma^i{}_{jk}(x) v^j v^k\Big\| \leq \sum_j \sum_k B_{ijk} \|v\|^2 \leq C\|v\|^2,$$

using `norm_sum_le`, `norm_mul`, and $|v^j| \leq \|v\|$ (`norm_le_pi_norm`). The componentwise bound is assembled into a norm bound via `pi_norm_le_iff_of_nonneg`. □

6.2.3 Local existence (`Geodesic/LocalExistence.lean`)

Theorem 6.3 (Existence via Picard–Lindelöf)**.** *For every initial state $z_0 \in \texttt{State}(n)$ and every smooth Christoffel field Γ, there exists $\varepsilon > 0$ and a curve γ on $[-\varepsilon, \varepsilon]$ satisfying $\texttt{IsCoordinateGeodesicOn}$.*

Proof. `exists_isCoordinateGeodesicOn_Icc_of_isPicardLindelof`: the geodesic spray is locally Lipschitz (it is smooth in x and polynomial in v), so the Picard–Lindelöf theorem (`IsPicardLindelof`) applies to the phase-space ODE. The theorem produces a solution on a closed interval $[-\varepsilon, \varepsilon]$ with derivative property at each t. `localCoordinateGeodesicFlowData` packages a uniform choice of ε, r, L, K for all initial states in a compact neighbourhood. □

6.2.4 Local uniqueness (`Geodesic/LocalUniqueness.lean`)

Theorem 6.4 (Uniqueness on open intervals)**.** *Let $t_0 \in (a, b)$ and let Γ be Lipschitz with constant K on (a, b). If γ, γ' both satisfy $\texttt{IsCoordinateGeodesicOn}$ on (a, b) and $\gamma(t_0) = \gamma'(t_0)$, then $\gamma = \gamma'$ on (a, b).*

Proof. `eqOn_Ioo_of_isCoordinateGeodesicOn`: apply `ODE_solution_unique_of_mem_Ioo`. The inputs are:

1. the Lipschitz condition on (a, b) lifted to all of $\texttt{State}(n)$ via `LipschitzWith.lipschitzOnWith`;
2. for each $t \in (a, b)$, the derivative property $\gamma'(t) = \texttt{geodesicVectorField}(\Gamma)(t, \gamma(t))$, promoted to a full `HasDerivAt` using `isOpen_Ioo.mem_nhds`;
3. the same for γ';
4. $\gamma(t_0) = \gamma'(t_0)$.

□

Theorem 6.5 (Uniqueness on closed intervals)**.** *If γ, γ' are geodesics on $[a, b]$ with $\gamma(t_0) = \gamma'(t_0)$ for $t_0 \in (a, b)$, and Γ is Lipschitz on (a, b), then $\gamma = \gamma'$ on $[a, b]$.*

Proof. `eqOn_Icc_of_isCoordinateGeodesicOn_lipschitzOn`: apply `ODE_solution_unique_of_mem_Icc` with the same inputs, plus: continuity on $[a, b]$ from `HasDerivWithinAt.continuousOn`, and interior derivative promotion via `Icc_mem_nhds` (which uses strict inequalities $t_0 \in (a, b)$ to get an open neighbourhood inside $[a, b]$). □

Corollary 6.6 (Uniqueness from matching position and velocity)**.** *If $\gamma(t_0)$ and $\gamma'(t_0)$ have equal positions and equal velocities (i.e., `statePosition` and `stateVelocity` match), then $\gamma = \gamma'$ on the open interval. The proof applies `Prod.ext` to reduce the equality of states to the componentwise equalities, then invokes Theorem 6.4.*

7 Exponential Map

Remark 7.1 (Proof status). The following files are fully proved in this section: `Basic`, `DifferentialAtZero`, `LocalInverse`, `RadialBridge`, `LocalRiemannianData` (`isMetricCompatible`), and the two theorems `gaussLemma_coordinate` and `exists_metricCon` from `GaussLemma`.

7.1 Main theorems

Main Theorem 7.1 (Exponential map: identity and differential at zero)**.** *Let Γ be a smooth Christoffel field and $p \in \mathbb{R}^n$. Define $\exp_p(v) := \gamma_v(1)$, where γ_v is the geodesic with $\gamma_v(0) = p$ and $\dot{\gamma}_v(0) = v$. Then:*

1. $\exp_p(0) = p$ (`coordinateExp_zero`)*;*
2. $D\exp_p(0) = \mathrm{id}$ (`fderiv (coordinateExp Gamma p) 0 = id`)*;*
3. $\exp_p$ *is a local diffeomorphism near* 0 (`LocalInverse.lean`)*.*

7.2 Proof architecture

7.2.1 Coordinate exponential map (`Exponential/Basic.lean`)

Definition 7.2 (Coordinate exponential map)**.** *The* geodesic family at base p *is the collection of geodesics $\gamma_v : [0,1] \to \mathbb{R}^n$ parameterized by initial velocity v.*

$$\texttt{coordinateExp}(\Gamma)(p)(v) := \gamma_v(1).$$

`coordinateExp_zero`: $\gamma_0 \equiv p$ (the constant curve), so `coordinateExp`$(\Gamma)(p)(0) = p$.

7.2.2 Differential at zero (`Exponential/DifferentialAtZero.lean`)

Theorem 7.3 (Identity differential)**.** $\mathrm{fderiv}(\texttt{coordinateExp}(\Gamma)(p))(0) = \mathrm{id}$.

Proof. The proof constructs an auxiliary *affine trial curve*:

$$\texttt{affineTrialCurve}(p, v)(t) := (p + t \cdot v,\ v).$$

Its derivative at any t is $(v, 0) \in \texttt{State}(n)$, computed by:

1. $\frac{d}{dt}(p + tv) = v$ via `hasDerivAt_id` composed with `smul_const` and `const_add`;
2. $\frac{d}{dt}(v) = 0$ via `hasDerivAt_const`;
3. combined via `HasDerivAt.prodMk`.

The geodesic acceleration is *quadratic* in velocity: $\|\texttt{geodesicAcceleration}(\Gamma)(x, v)\| \le C\|v\|^2$ (Lemma 6.2). Therefore, along the trial curve (where v is the initial velocity), the geodesic error satisfies

$$\|\texttt{geodesicSpray}(\Gamma)(\texttt{affineTrialCurve}(p, v)(t)) - \frac{d}{dt}\texttt{affineTrialCurve}(p, v)(t)\| = \|(\text{accel term}, 0)\| \le C\|v\|^2.$$

The Fréchet derivative characterization then gives

$$\|\texttt{coordinateExp}(\Gamma)(p)(v) - p - v\| = o(\|v\|),$$

since the geodesic deviates from $p+v$ (the endpoint of the trial curve) by at most $O(\|v\|^2)$ via Gronwall's inequality. This identifies the derivative as the identity map $v \mapsto v$. In Lean, this is packaged using `Asymptotics.IsLittleO` and `HasFDerivAt`. □

7.2.3 Local inverse via the Inverse Function Theorem (`Exponential/LocalInverse.lean`)

Theorem 7.4 (Local diffeomorphism)**.** *Given a C^1 exponential map `exp` with D`exp`$(0) = \mathrm{id}$ (in the strict sense: `StrictDifferentialA` the exponential map is a local homeomorphism near the origin.*

Proof. `localExpPartialHomeomorphOfStrict`: apply `HasStrictFDerivAt.toOpenPartialHomeomorph` to the strict derivative D`exp`$(0) = \mathrm{id}$ (packed as a continuous linear equiv from `velocityPositionEquiv`). Mathlib's IFT produces an `OpenPartialHomeomorph` whose source contains 0 (`zero_mem_localExp_source_ofStrict`) and whose target contains `exp`(0) (`base_mem_localExp_target_ofSt`

The left inverse (`localExp_leftInverse_ofStrict`) and right inverse (`localExp_rightInverse_ofStrict`) are the left/right inverse identities of the `PartialHomeomorph`. At the base point, `localExpInverse`(`exp`)$(p) = 0$ is proved by rewriting $p =$ `exp`(0) and applying `PartialHomeomorph.left_inv`. □

7.2.4 Radial source (`Exponential/RadialBridge.lean`)

Definition 7.5 (Radial source)**.** *The* radial source *is the star-shaped domain* `radialSource`$(\Gamma, p) \subseteq \mathbb{R}^n$ *on which the geodesic flow is defined for time* $[0, 1]$*. It is an open set containing* 0*.*

Theorem 7.6 (Star-shapedness)**.** *If* $v \in$ `radialSource`(Γ, p) *and* $t \in [0, 1]$*, then* $t \cdot v \in$ `radialSource`(Γ, p)*.*

Proof. `smul_mem_radialSource_of_mem`: let ε, r be the constants from `LocalCoordinateGeodesicFlowData`. The membership condition is $\|v\| < \varepsilon r$. Then

$$\|t \cdot v\| = |t|\,\|v\| \le \|v\| < \varepsilon r,$$

because $|t| \le 1$ from $t \in [0, 1]$ (and $|t| \le 1$ is derived from $t \in$ `Icc 0 1` via `abs_of_nonneg`), and $|t|\,\|v\| \le \|v\|$ by `nlinarith`. □

7.2.5 Gauss lemma at the origin (`Exponential/GaussLemma.lean`)

Theorem 7.7 (`gaussLemma_coordinate`)**.** *At the base point* p*, for all* v *in the source:*

$$\sum_{i,j} g_{ij}(p) \cdot (\exp_p v)^i \cdot v^j = \sum_{i,j} g_{ij}(p) \cdot v^i \cdot v^j.$$

Proof. Differentiate the speed-squared $s(t) = \sum_{ij} g(\gamma(t))_{ij} \dot\gamma^i(t) \dot\gamma^j(t)$ along the geodesic $\gamma(t) = \exp_p(tv)$ and evaluate at $t = 0$. Metric compatibility gives $\dot s(t) = 0$ (the geodesic has constant speed). Setting $t = 0$: $\dot\gamma(0) = v$, so $s(0) = \sum_{ij} g_{ij}(p)\, v^i\, v^j$. Setting $t = 1$: $\dot\gamma(1) =?$; the pairing $\sum_{ij} g_{ij}(p)(\exp_p v)^i v^j$ is the limit of the radial pairing at $t = 0$. In Lean, the proof applies the chain rule and Gronwall to control the integral of $\partial_t \big[\sum_{ij} g_{ij}(\gamma) \dot\gamma^i v^j\big]$. □

Theorem 7.8 (`exists_metricCompatible_coordinate_middle_formula`)**.** *For any Christoffel-compatible metric and radial geodesic* $\gamma(t) = \exp_p(tv)$*,*

$$\partial_t \Big[\sum_{ij} g_{ij}(\gamma(t))\, \dot\gamma^i(t)\, v^j\Big] = 0.$$

Proof. Apply the product rule and expand $\partial_t g_{ij}(\gamma)$ using the chain rule: $\partial_t g_{ij}(\gamma) = \sum_k (\partial_k g_{ij})(\gamma) \dot\gamma^k$. The Christoffel compatibility condition $\partial_k g_{ij} = g_{il} \Gamma^l{}_{jk} + g_{jl} \Gamma^l{}_{ik}$ turns the derivative into connection terms. The geodesic equation $\ddot\gamma^i = -\Gamma^i{}_{jk} \dot\gamma^j \dot\gamma^k$ kills the remaining terms via a cancellation. The Lean proof invokes `metric_contract_leviCivitaChristoffel` and `leviCivitaChristoffel_lower_` □

8 Local Minimization: Normal Neighborhoods

Remark 8.1 (Proof status). One file in this module is fully proved: `Minimization/NormalNeighborhoods.lean`. The remaining files (`LocalMinimizers`, `MetricBridge`, `ShortGeodesicsMinimize`) are conditionally proved pending the two remaining owner `sorry`s in `Exponential/RadialGaussLemma` and `Minimization/MetricBridge`, and are not covered here.

8.1 Normal neighborhoods (`Minimization/NormalNeighborhoods.lean`)

Definition 8.2 (Normal neighborhood)**.** *For a smooth Christoffel field* Γ *and a base point* $p \in \mathbb{R}^n$*, the* normal neighborhood *of* p *is the target of the coordinate exponential chart:*

$$\mathit{normalNeighborhood}(\Gamma, p) := (\mathit{coordinateExpPartialHomeomorph}(\Gamma, p)).\mathit{target} \subseteq \mathbb{R}^n.$$

Equivalently, it is the image of the exponential source under $\exp_p$*.*

Theorem 8.3 (`normalNeighborhood_open`)**.** *The normal neighborhood* $\mathit{normalNeighborhood}(\Gamma, p)$ *is open.*

Proof. The target of any `PartialHomeomorph` is open by definition (`PartialHomeomorph.open_target`). The coordinate exponential chart is a `PartialHomeomorph` by construction in `Exponential/LocalInverse.lean`, so its target is open. □

Theorem 8.4 (`base_mem_normalNeighborhood`)**.** *The base point* p *lies in its own normal neighborhood.*

Proof. `Exponential.Coordinate.base_mem_coordinateExpPartialHomeomorph_target`: by the local inverse theorem, $p = \exp_p(0)$ lies in the target of the exponential chart, since 0 lies in the source. □

Theorem 8.5 (`coordinateExp_normalCoordinates_eq`)**.** *On the normal neighborhood, the composition* $\exp_p \circ \mathrm{normalCoords}_p = \mathrm{id}$*: for every* $x \in \mathit{normalNeighborhood}(\Gamma, p)$*,*

$$\exp_p\big(\mathrm{normalCoords}_p(x)\big) = x.$$

Proof. This is the right-inverse identity of the `PartialHomeomorph`: $\varphi \circ \varphi^{-1} = \mathrm{id}$ on the target. In Lean, `coordinateExp_normalCoordi` states the `Set.EqOn` identity, and the proof `coordinateExp_normalCoordinates_eq` wraps it as a pointwise equality. □

Theorem 8.6 (`normalCoordinates_coordinateExp_eq`)**.** *On the chart source, the normal-coordinates map inverts the exponential: for every* v *in the source of the exponential chart,*

$$\mathrm{normalCoords}_p\big(\exp_p(v)\big) = v.$$

Proof. This is the left-inverse identity of the `PartialHomeomorph`: $\varphi^{-1} \circ \varphi = \mathrm{id}$ on the source, given by `normalCoordinates_coordinate` in `Exponential/NormalCoordinates.lean`. □

9 Riemannian Volume and Polar Coordinates

Remark 9.1 (Proof status). `Measure/PolarCoordinates` is **fully proved** — no `sorry`, no external inputs. `Measure/RiemannianVolume` provides foundational structures; `Measure/ExpJacobian` and `Measure/PullbackDensity` contain active boundary resolution.

9.1 Main result

Main Theorem 9.1 (Polar decomposition via the exponential map)**.** *Let* $\exp_p : U \to \mathbb{R}^n$ *be a local exponential map with* $U \subseteq \mathbb{R}^n$ *a star-shaped neighbourhood of* 0*. Then the map*

$$\varphi : U \setminus \{0\} \to (0, \infty) \times S^{n-1}, \qquad v \mapsto (\|v\|,\ v/\|v\|),$$

is a diffeomorphism, and the composition $\exp_p \circ \varphi^{-1}$ *provides a polar decomposition of a neighbourhood of* p*. In particular,*

$$(\exp_p)^* \, dV_g = J(r, \omega) \, dr \, d\sigma_{n-1}(\omega),$$

where $J(r, \omega) = |\det D \exp_p(r\omega)|$*.*

9.2 Proof architecture

9.2.1 Riemannian volume (`Measure/RiemannianVolume.lean`)

Definition 9.2 (Riemannian volume density)**.** *Given a smooth positive-definite metric field* $g : \mathbb{R}^n \to \mathrm{Mat}(n, \mathbb{R})$*, the* Riemannian volume density *at* x *is*

$$\rho_g(x) := \sqrt{\det g(x)}.$$

`MetricField` packages g *with its smoothness and positive-definiteness. `densityOfTensor` computes* ρ_g *and `IsDensity` asserts measurability and positivity. The Riemannian volume measure is* $dV_g = \rho_g \cdot d\mathcal{L}^n$*, where* $\mathcal{L}^n$ *is the Lebesgue measure on* $\mathbb{R}^n$*.*

9.2.2 Polar decomposition (`Measure/PolarCoordinates.lean`)

Definition 9.3 (Polar decomposition structure)**.** *`HasPolarDecomposition`*(f, U) *is the predicate asserting that a smooth map* $f : U \setminus \{0\} \to (0, \infty) \times S^{n-1}$ *is a volume-preserving diffeomorphism: specifically that* f *is a smooth bijection with the Jacobian formula* $f^*(dr \wedge d\sigma) = dv$*.*

Theorem 9.4 (`hasPolarDecomposition_coordinateExpLocalExponentialMap`)**.** *Let* Γ *be a smooth Christoffel field and let* $U =$ *`radialSource`*(Γ, p)*. The standard polar map* $v \mapsto (\|v\|, v/\|v\|)$ *is a smooth diffeomorphism on* $U \setminus \{0\}$*, and its composition with the inverse exponential map provides a polar coordinate chart for a neighbourhood of* p*.*

Proof. The proof proceeds in three steps.

Step 1: star-shapedness. By `smul_mem_radialSource_of_mem` (Theorem 7.6), the radial source U is star-shaped: for any $v \in U$ and $t \in [0, 1]$, $tv \in U$. This means the entire segment from 0 to v lies in U.

Step 2: local diffeomorphism structure. The local exponential map $\exp_p : U \to \mathbb{R}^n$ is a local diffeomorphism by `localExpPartialHomeomorph` (Theorem 7.4). Its Jacobian at 0 is the identity (`gaussLemma_coordinate`).

Step 3: polar coordinate compatibility. The map $(r, \omega) \mapsto \exp_p(r\omega)$ is smooth on $(0, \varepsilon) \times S^{n-1}$ by composition of the smooth exponential map with the smooth embedding $r \cdot \omega : (0, \infty) \times S^{n-1} \to \mathbb{R}^n$. The Jacobian determinant of this composition equals $J(r, \omega) \cdot r^{n-1}$, where r^{n-1} comes from the standard polar Jacobian of $\mathbb{R}^n$. The `HasPolarDecomposition` structure is verified by checking the Jacobian identity using the chain rule and the polar metric formula from `gaussLemma_coordinate`. □

9.2.3 Pullback density (`Measure/PullbackDensity.lean`)

Definition 9.5 (Pullback measure)**.** *For a smooth map* $\varphi : U \to V$ *between open subsets of* $\mathbb{R}^n$ *and a measure* μ *on* V *with density* ρ*, the* pullback measure $\varphi^*\mu$ *has density*

$$(\varphi^*\rho)(x) := |\det D\varphi(x)| \cdot \rho(\varphi(x)).$$

`pullbackDensity` and `pullbackDensityENNReal` package this computation; `pullbackMeasure` assembles the measure using Mathlib's `MeasureTheory.Function.Jacobian` theorem.

9.2.4 Exponential map Jacobian (`Measure/ExpJacobian.lean`)

Definition 9.6 (Exponential Jacobian function)**.** *`ExpJacobian`*$(r, \omega) := |\det D \exp_p(r\omega)|$*.*

Two bounds on the Jacobian are established without external witnesses:

- `expJacobianFn_le_prod_column_norms`: Hadamard's determinant bound $|\det A| \le \prod_i \|Ae_i\|$ applied to $D \exp_p$.
- `expJacobianFn_le_of_basiswise_bound`: if each column of $D \exp_p$ is bounded in norm, the Hadamard bound propagates to a pointwise Jacobian bound, used downstream in volume comparison.

10 Variation Formulas

10.1 Main theorem

The variation branch formalizes the structural identity

$$\dot{\mathcal{F}} = d_\nabla \dot{A},$$

then specializes it to the Levi-Civita variation formula and contracts to Ricci and scalar curvature.

Main Theorem 10.1 (Scalar variation formula in the formalized coordinate package)**.** *Assume a coordinate variation package D, a connection form A, an inverse metric g^{-1}, a metric variation h, a background Ricci tensor* Ric*, and a covariant derivative package ∇h satisfying the Levi-Civita bridge*

$$\dot{\Gamma}^k{}_{ij} = \frac{1}{2}\sum_\ell g^{k\ell}\big(\nabla_i h_{j\ell} + \nabla_j h_{i\ell} - \nabla_\ell h_{ij}\big).$$

Then the formalized scalar variation splits as

$$\dot{S} = -\langle h, \mathrm{Ric}\rangle + \mathrm{tr}_g(\dot{\mathrm{Ric}}) = -\sum_{j,\nu} h_{j\nu}\,\mathrm{Ric}_{j\nu} + \sum_{j,\nu} g^{j\nu}\dot{\mathrm{Ric}}_{j\nu},$$

where the Ricci variation $\dot{\mathrm{Ric}}_{j\nu}$ itself is obtained by contracting the curvature variation formula

$$\dot{R} = d\dot{A} + \dot{A}\wedge A + A\wedge \dot{A}.$$

Remark 10.1. The last Lean theorem is intentionally kept at the level of explicit finite sums. It packages the standard scalar variation formula into two pieces: the inverse-metric variation term and the contraction of the Ricci variation.

10.2 Proof architecture

The variation branch has the same layered structure as the previous sections:

1. abstract one-parameter connection algebra;
2. component bridge and coordinate curvature variation;
3. Levi-Civita bridge for $\dot{\Gamma}$;
4. Ricci and scalar contractions.

10.3 Abstract one-parameter connection calculus (`Variation/Basic.lean`, `Variation/Variation.lean`)

Definition 10.2 (Local connection variation algebra)**.** *A local connection variation algebra consists of groups Ω^1, Ω^2, maps*

$$d_1 : \Omega^1 \to \Omega^2, \qquad \wedge_{11} : \Omega^1 \times \Omega^1 \to \Omega^2,$$

and formal variation operators

$$\partial_1 : \Omega^1 \to \Omega^1, \qquad \partial_2 : \Omega^2 \to \Omega^2,$$

satisfying additivity of ∂_2, compatibility

$$\partial_2(d_1 A) = d_1(\partial_1 A),$$

and the product rule

$$\partial_2(A \wedge_{11} B) = (\partial_1 A)\wedge_{11} B + A \wedge_{11} (\partial_1 B).$$

The file then defines

$$\mathcal{F}(A) := d_1 A + A \wedge_{11} A, \qquad \dot{A} := \partial_1 A,$$

and

$$d_\nabla \dot{A} := d_1 \dot{A} + \dot{A} \wedge_{11} A + A \wedge_{11} \dot{A}.$$

Theorem 10.3 (Abstract curvature variation identity)**.** *For every connection form A,*

$$\partial_2(\mathcal{F}(A)) = d_\nabla \dot{A}.$$

Proof. Expand the curvature:

$$\partial_2(\mathcal{F}(A)) = \partial_2(d_1 A + A \wedge_{11} A).$$

By additivity and the two derivation axioms,

$$\partial_2(\mathcal{F}(A)) = d_1(\partial_1 A) + (\partial_1 A)\wedge_{11} A + A \wedge_{11} (\partial_1 A),$$

which is exactly $d_\nabla \dot{A}$. The Lean proof is the now-familiar pattern: `simp`, then rewrite by the derivation laws, then `abel`. □

10.4 Component bridge and coordinate curvature variation (`Variation/ComponentBridge.lean`, `Variation/CoordinateVariation.lean`)

The coordinate layer uses

$$\Omega_n^1 = (\mathrm{Fin}\, n)^3 \to \mathbb{R}, \qquad \Omega_n^2 = (\mathrm{Fin}\, n)^4 \to \mathbb{R},$$

with the same matrix-valued wedge product as in the Second Bianchi branch:

$$(A \wedge_{11} B)^i{}_{j,\mu\nu} = \sum_k \big(A^i{}_{k,\mu} B^k{}_{j,\nu} - A^i{}_{k,\nu} B^k{}_{j,\mu}\big).$$

Theorem 10.4 (Coordinate curvature variation)**.** *If $\dot{A} = \partial_1 A$, then*

$$\dot{R}^i{}_{j,\mu\nu} = (d_1 \dot{A})^i{}_{j,\mu\nu} + \sum_k \big(\dot{A}^i{}_{k,\mu} A^k{}_{j,\nu} - \dot{A}^i{}_{k,\nu} A^k{}_{j,\mu}\big) + \sum_k \big(A^i{}_{k,\mu} \dot{A}^k{}_{j,\nu} - A^i{}_{k,\nu} \dot{A}^k{}_{j,\mu}\big).$$

Proof. Project the abstract identity of Theorem 10.3 to the (i, j, μ, ν)-component. The theorem `proj_curvature_variation_eq_expli` unfolds the two wedge terms, and `classical_curvature_variation` is exactly the resulting formula. □

10.5 Levi-Civita bridge (`Variation/LeviCivitaVariationBridge.lean`)

The next file introduces the standard coordinate variation of the Levi-Civita connection:

$$\dot{\Gamma}^k{}_{ij} = \frac{1}{2} \sum_\ell g^{k\ell} \big(\nabla_i h_{j\ell} + \nabla_j h_{i\ell} - \nabla_\ell h_{ij}\big).$$

This is formalized as `leviCivitaVariation`.

Theorem 10.5 (Curvature variation under the Levi-Civita bridge)**.** *If $\dot{A}$ is identified with the Levi-Civita variation above, then the coordinate curvature variation formula of Theorem 10.4 specializes to the same formula with $\dot{A}$ replaced everywhere by $\dot{\Gamma}$.*

Proof. This is a direct substitution theorem. The Lean proof is a single `simpa` using `classical_curvature_variation` and the bridge hypothesis `hdotA`. □

10.6 Ricci and scalar contractions (`Variation/RicciScalarVariation.lean`)

The final file introduces the contractions

$$\dot{\mathrm{Ric}}_{j\nu} := \sum_i \dot{R}^i{}_{j,i\nu},$$

and

$$\dot{S} := -\sum_{j,\nu} h_{j\nu}\, \mathrm{Ric}_{j\nu} + \sum_{j,\nu} g^{j\nu} \dot{\mathrm{Ric}}_{j\nu}.$$

Theorem 10.6 (Ricci variation by contraction)**.** *The Ricci variation is obtained from the curvature variation formula by contracting the first and third curvature indices:*

$$\dot{\mathrm{Ric}}_{j\nu} = \sum_i \Big((d_1 \dot{A})^i{}_{j,i\nu} + \sum_k (\dot{A}^i{}_{k,i} A^k{}_{j,\nu} - \dot{A}^i{}_{k,\nu} A^k{}_{j,i}) + \sum_k (A^i{}_{k,i} \dot{A}^k{}_{j,\nu} - A^i{}_{k,\nu} \dot{A}^k{}_{j,i}) \Big).$$

Proof. This is precisely `ricciVariationFromCurvature` applied to Theorem 10.4. The Lean proof is a pointwise `Finset.sum_congr`. □

Theorem 10.7 (Scalar variation split)**.** *The scalar variation decomposes as*

$$\dot{S} = -\sum_{j,\nu} h_{j\nu}\, \mathrm{Ric}_{j\nu} + \sum_{j,\nu} g^{j\nu} \dot{\mathrm{Ric}}_{j\nu},$$

and after substituting Theorem 10.6 this becomes the explicit double-sum formula formalized in `scalar_variation_formula`.

Proof. The first equality is built into the definition `scalarVariationFormulaValue`. The second is obtained by replacing $\dot{\mathrm{Ric}}_{j\nu}$ with its contracted curvature-variation expression. The Lean proof again works pointwise over the finite sums and rewrites by `ricci_variation_formula`. □

Finally, `scalar_variation_formula_of_leviCivita_bridge` substitutes the Levi-Civita bridge into Theorem 10.7, giving the fully formalized coordinate scalar variation formula advertised in Theorem 10.1.

11 Jacobi Fields

11.1 Main theorem

Main Theorem 11.1 (Existence and uniqueness of Jacobi fields)**.** *Let $T : \mathbb{R} \to \mathbb{R}^n$ be the tangent field of a geodesic, let R be a curvature tensor, and let $A : \mathbb{R} \to \mathrm{Mat}(n \times n, \mathbb{R})$ be the matrix $A(t) = [R(\cdot, T(t))T(t)]$. For every initial data $(J_0, J_0') \in \mathbb{R}^n \times \mathbb{R}^n$, there exists a unique smooth field $J : \mathbb{R} \to \mathbb{R}^n$ satisfying*

$$J''(t) + A(t)\,J(t) = 0, \qquad J(0) = J_0, \quad J'(0) = J_0'.$$

11.2 Proof architecture

11.2.1 Abstract Jacobi field predicate (`Jacobi/Basic.lean`)

Definition 11.1 (Jacobi field)**.** *Given a curvature tensor R, a tangent field T, a second covariant derivative $\nabla_T \nabla_T$, and a field J along the geodesic, the predicate `IsJacobiField` asserts:*

$$\forall t, \quad (\nabla_T \nabla_T J)(t) + R(J(t), T(t))T(t) = 0.$$

`InitialData` packages the pair $(J_0, J_0') = (J(0), (\nabla_T J)(0))$.

Definition 11.2 (Jacobi operator)**.** *A `JacobiOperator` is a map $\mathcal{J}$: `InitialData` $\to$ `FieldAlong`. The specification `IsJacobiOperato` requires that for every initial datum, $\mathcal{J}(\mathrm{data})$ satisfies `IsJacobiField` with the correct initial values, and that any other field with the same initial data and Jacobi property equals $\mathcal{J}(\mathrm{data})$.*

11.2.2 First-order system reduction (`Jacobi/ExistenceUniqueness.lean`)

Definition 11.3 (Jacobi vector field and state space)**.** *The Jacobi equation $J''+A(t)J = 0$ is lifted to the state space `JacobiState`$(n) = \mathbb{R}^n \times \mathbb{R}^n$ by setting $Z = (U, V) = (J, J')$ and defining*

$$\texttt{jacobiVectorField}(\mathrm{sys}, t, Z) := (Z_2,\ \texttt{coordinateParallelRhs}(A, t, Z_1)) = (V,\ -A(t)U).$$

The `CoordinateJacobiSystem` packages the matrix family $A : \mathbb{R} \to \mathrm{Mat}(n, \mathbb{R})$ together with the continuity hypothesis $\forall i, j,\ t \mapsto A(t)_{ij}$ continuous.

Theorem 11.4 (Jacobi vector field is Lipschitz)**.** *If $\|\texttt{coordinateCoeffBound}(A(t))\| \le K$ and $K \ge 1$, then `jacobiVectorField`$(\mathrm{sys}, t, \cdot$ is Lipschitz with constant K.*

Proof. `lipschitzWith_jacobiVectorField` applies `LipschitzWith.of_dist_le_mul`. For $Z = (x, u)$ and $W = (y, v)$, the product-space distance is $\max(d(x, y), d(u, v))$. The second component `coordinateParallelRhs`$(A, t, \cdot)$ is Lipschitz with constant K (by `lipschitzWith_coordinateParallelRhs`). The first component contributes $d(u, v)$. Using $K \ge 1$, both components are bounded by $K \cdot d(Z, W)$ via a short `calc` block and `nlinarith`. □

Theorem 11.5 (Local existence and uniqueness of the Jacobi state system)**.** *For every initial state (J_0, J_0') near the origin and every $\varepsilon > 0$, there exists a unique solution $Z : [-\varepsilon, \varepsilon] \to \mathbb{R}^{2n}$ of the Jacobi system with $Z(0) = (J_0, J_0')$.*

Proof. `existsUnique_coordinateJacobiState` proceeds in two parts.

Existence is immediate from the Picard–Lindelöf hypothesis `hpl.exists_eq_forall_mem_Icc_hasDerivWithinAt`: the hypothesis packages the Lipschitz condition (Theorem 11.4) and the initial ball constraint, and Mathlib's `IsPicardLindelof` gives a solution $Z : [-\varepsilon, \varepsilon] \to \mathbb{R}^{2n}$ with $Z(0) = (J_0, J_0')$ and the derivative property on each sub-interval.

Uniqueness. Let W be another solution with $W(0) = (J_0, J_0')$. To show $W = Z$ on $[-\varepsilon, \varepsilon]$, apply `ODE_solution_unique_of_mem_Ic`

1. W and Z are continuous on $[-\varepsilon, \varepsilon]$ (by `ContinuousWithinAt.continuousOn` from the derivative property).
2. Both have the correct derivatives on $(-\varepsilon, \varepsilon)$ (by `HasDerivWithinAt.hasDerivAt` using the open-set neighbourhood `Icc_mem_nhds`
3. The vector field is Lipschitz on all of $\mathbb{R}^{2n}$ at each t (by `lipschitzWith_jacobiVectorField.lipschitzOnWith`).
4. $W(0) = Z(0)$ (by the initial condition and the stored $Z(0)$ formula `coordinateInitialPoint`).

These four inputs are fed to `ODE_solution_unique_of_mem_Icc`, which applies Gronwall's inequality to the difference $\|W(t) - Z(t)\|$ and concludes equality on the full interval. □

12 Second Bianchi Identity

12.1 Main theorem

The original natural-language document formalized the following theorem, and the present version keeps that proof as the first major section of the consolidated write-up.

Main Theorem 12.1 (Second Bianchi identity in coordinates)**.** *Let $n \geq 1$ and let ${A^i}_{j,k}$ be a smooth torsion-free connection on $\mathbb{R}^n$, so ${A^i}_{j,k}(x) = {A^i}_{k,j}(x)$ for all x, i, j, k. Define the curvature by*

$$ {R^i}_{j,kl} = \partial_k {A^i}_{j,l} - \partial_l {A^i}_{j,k} + \sum_m \big({A^i}_{m,k} {A^m}_{j,l} - {A^i}_{m,l} {A^m}_{j,k} \big). $$

Then for every $x \in \mathbb{R}^n$ and all indices i, j, e, k, l,

$$ D_e {R^i}_{j,kl} + D_k {R^i}_{j,le} + D_l {R^i}_{j,ek} = 0, $$

where

$$ D_e {R^i}_{j,kl} := \partial_e {R^i}_{j,kl} + \sum_m \big({A^i}_{m,e} {R^m}_{j,kl} - {R^i}_{m,kl} {A^m}_{j,e} - {A^m}_{k,e} {R^i}_{j,ml} - {A^m}_{l,e} {R^i}_{j,km} \big). $$

Remark 12.1. This is the classical cyclic identity $D_{[e} {R^i}_{jkl]} = 0$. The Lean package proves it by separating the abstract algebraic identity $d_\nabla \mathcal{F} = 0$ from the coordinate and smooth-analytic bridge statements.

12.2 Proof architecture

The formal proof is organized in four layers:

1. **Abstract algebra** in `SecondBianchi/Basic.lean` and `SecondBianchi/SecondBianchi.lean`.
2. **Component extraction** in `SecondBianchi/ComponentBridge.lean`.
3. **Coordinate bridge** in `SecondBianchi/CoordinateBianchi.lean`, `SecondBianchi/ClassicalCoordinateBianchi.lean`, and `SecondBianchi/ConcreteCoordinateBianchi.lean`.
4. **Smooth instantiation** in `SecondBianchi/SmoothCoordinateScaffold.lean` and `SecondBianchi/SmoothCoordinateBianchi`

12.3 Abstract local connection algebra (`SecondBianchi/Basic.lean`)

Definition 12.2 (Local connection algebra)**.** *A* local connection algebra *consists of abelian groups $\Omega^1, \Omega^2, \Omega^3$, maps*

$$ d_1 : \Omega^1 \to \Omega^2, \qquad d_2 : \Omega^2 \to \Omega^3, $$

and wedge operations

$$ \wedge_{11} : \Omega^1 \times \Omega^1 \to \Omega^2, \quad \wedge_{12} : \Omega^1 \times \Omega^2 \to \Omega^3, \quad \wedge_{21} : \Omega^2 \times \Omega^1 \to \Omega^3, $$

satisfying additivity, $d_2 \circ d_1 = 0$, the Leibniz rule

$$ d_2(\alpha \wedge_{11} \beta) = (d_1 \alpha) \wedge_{21} \beta - \alpha \wedge_{12} (d_1 \beta), $$

and the associativity relation

$$ \alpha \wedge_{12} (\beta \wedge_{11} \gamma) = (\alpha \wedge_{11} \beta) \wedge_{21} \gamma. $$

Definition 12.3 (Curvature and covariant exterior derivative)**.** *For a connection form $\nabla \in \Omega^1$ define*

$$ \mathcal{F}(\nabla) := d_1 \nabla + \nabla \wedge_{11} \nabla, \qquad d_\nabla \omega := d_2 \omega + \nabla \wedge_{12} \omega - \omega \wedge_{21} \nabla. $$

12.4 Abstract second Bianchi identity (`SecondBianchi/SecondBianchi.lean`)

Theorem 12.4 (Abstract second Bianchi identity)**.** *For every local connection algebra and every connection form ∇,*

$$ d_\nabla(\mathcal{F}(\nabla)) = 0. $$

Proof. Expand both definitions:

$$ d_\nabla(\mathcal{F}(\nabla)) = d_2(d_1 \nabla + \nabla \wedge_{11} \nabla) + \nabla \wedge_{12} (d_1 \nabla + \nabla \wedge_{11} \nabla) - (d_1 \nabla + \nabla \wedge_{11} \nabla) \wedge_{21} \nabla. $$

Now use additivity of d_2, the identity $d_2(d_1 \nabla) = 0$, and the Leibniz rule to rewrite the first term as

$$ (d_1 \nabla) \wedge_{21} \nabla - \nabla \wedge_{12} (d_1 \nabla). $$

Distributing the remaining wedge terms gives six summands. Two cancel immediately, and the last two cancel by associativity:

$$ \nabla \wedge_{12} (\nabla \wedge_{11} \nabla) = (\nabla \wedge_{11} \nabla) \wedge_{21} \nabla. $$

The Lean proof is the same four-line pattern as in the source: `simp`; then two `rw` blocks; then `abel`. □

12.5 Component bridge (`SecondBianchi/ComponentBridge.lean`)

Definition 12.5 (Component projection)**.** *A component projection is a map extracting a chosen coefficient of a 3-form. Formally it is a map $P : \Omega^3 \to I \to C$ with $P(0)(\iota) = 0$.*

Theorem 12.6 (Componentwise vanishing)**.** *For every component projection P and every connection form ∇,*

$$P(d_\nabla(\mathcal{F}(\nabla)))(\iota) = 0$$

for all indices $\iota \in I$.

Proof. Apply the projection to Theorem 12.4 and use that the projection of 0 is 0. □

12.6 Coordinate instantiation (`CoordinateBianchi.lean`, `ClassicalCoordinateBianchi.lean`, `ConcreteC`

The coordinate model uses

$$\Omega^1_n = (\mathrm{Fin}\, n)^3 \to \mathbb{R}, \qquad \Omega^2_n = (\mathrm{Fin}\, n)^4 \to \mathbb{R}, \qquad \Omega^3_n = (\mathrm{Fin}\, n)^5 \to \mathbb{R},$$

with the first two slots carrying the matrix indices and the last slots carrying the alternating form indices.

The coordinate wedge products are

$$\begin{aligned}
(A \wedge_{11} B)^i{}_{j,\mu\nu} &= \sum_k \big(A^i{}_{k,\mu} B^k{}_{j,\nu} - A^i{}_{k,\nu} B^k{}_{j,\mu}\big),\\
(A \wedge_{12} \Omega)^i{}_{j,ekl} &= \sum_m \big(A^i{}_{m,e} \Omega^m{}_{j,kl} + A^i{}_{m,k} \Omega^m{}_{j,le} + A^i{}_{m,l} \Omega^m{}_{j,ek}\big),\\
(\Omega \wedge_{21} A)^i{}_{j,ekl} &= \sum_m \big(\Omega^i{}_{m,ek} A^m{}_{j,l} + \Omega^i{}_{m,kl} A^m{}_{j,e} + \Omega^i{}_{m,le} A^m{}_{j,k}\big).
\end{aligned}$$

Theorem 12.7 (Coordinate component form)**.** *If $\mathcal{F} = d_1 A + A \wedge_{11} A$, then for all i, j, e, k, l,*

$$(d_2\mathcal{F})^i{}_{j,ekl} + \sum_m \big(A^i{}_{m,e} \mathcal{F}^m{}_{j,kl} + A^i{}_{m,k} \mathcal{F}^m{}_{j,le} + A^i{}_{m,l} \mathcal{F}^m{}_{j,ek}\big) - \sum_m \big(\mathcal{F}^i{}_{m,ek} A^m{}_{j,l} + \mathcal{F}^i{}_{m,kl} A^m{}_{j,e} + \mathcal{F}^i{}_{m,le} A^m{}_{j,k}\big) = 0.$$

Proof. The component projection sends a 3-form to its (i, j, e, k, l)-entry. Unfolding the coordinate wedge products identifies the component of $d_\nabla(\mathcal{F})$ with the displayed formula, so Theorem 12.6 gives the result. □

The classical bridge file then rewrites the same expression as the cyclic covariant derivative $D_{[e} R^i{}_{j,kl]}$, using three facts:

1. d_2 is the cyclic sum of partial derivatives,
2. the connection is torsion-free,
3. the curvature is antisymmetric in its last two indices.

The extra form-index terms cancel pairwise, and the Lean proof finishes with `abel_nf` and `linarith`. The concrete file supplies explicit coordinate formulas for d_1 and d_2, proves curvature antisymmetry, and deduces the concrete cyclic Bianchi identity.

12.7 Smooth instantiation (`SmoothCoordinateScaffold.lean`, `SmoothCoordinateBianchi.lean`)

The smooth layer replaces raw coordinate tensors by smooth coefficient fields on $\mathbb{R}^n$, defines d_1 and d_2 by Fréchet directional derivatives, and proves the two analytic axioms:

1. $d_2(d_1 A) = 0$, by symmetry of mixed second derivatives;
2. the Leibniz rule for $d_2(A \wedge_{11} B)$, by the product rule.

Once these are available, the abstract theorem and the coordinate bridge apply verbatim.

Theorem 12.8 (Smooth second Bianchi identity)**.** *For every smooth torsion-free connection on $\mathbb{R}^n$,*

$$D_e R^i{}_{j,kl}(x) + D_k R^i{}_{j,le}(x) + D_l R^i{}_{j,ek}(x) = 0$$

for all x, i, j, e, k, l.

13 Bochner Identity

13.1 Main theorem

The Bochner branch formalizes the scalar identity in a coordinate package built from a covector-valued commutator theorem and a trace step.

Main Theorem 13.1 (Concrete coordinate Bochner identity)**.** *Let D be coordinate connection data, let A be a connection 1-form, let α be a covector field, let H be its Hessian model, and let T be a third-covariant-derivative model. Assume*

$$[\nabla_\mu, \nabla_\nu]\alpha_k = T_{\mu\nu k} - T_{\nu\mu k},$$

and assume symmetry in the last two slots,

$$T_{ijk} = T_{ikj}.$$

Then Lean proves

$$\frac{1}{2}\,\Delta|\alpha|^2 = \|H\|^2 + \langle \alpha, \beta\rangle + \mathrm{Ric}(\alpha, \alpha),$$

where

$$\beta_j := \sum_i T_{jii}.$$

More precisely, the concrete formalization writes

$$\Delta|\alpha|^2 = 2\|H\|^2 + 2\langle \alpha, \nabla^*\nabla\,\alpha\rangle, \qquad (\nabla^*\nabla\,\alpha)_j := \sum_i T_{iij},$$

and proves the displayed identity above.

Remark 13.1. If $\alpha = \nabla f$, $H = \nabla^2 f$, and $T = \nabla^3 f$, then $\beta = \nabla\Delta f$, and this is the classical Bochner formula

$$\frac{1}{2}\Delta|\nabla f|^2 = |\nabla^2 f|^2 + \langle \nabla f, \nabla\Delta f\rangle + \mathrm{Ric}(\nabla f, \nabla f).$$

13.2 Proof architecture

The Bochner branch follows the roadmap

$$d_\nabla^2 = \mathcal{F} \wedge (\cdot) \Longrightarrow [\nabla_i, \nabla_j]\alpha_k = R_{ijk}{}^{\ell}\alpha_\ell \Longrightarrow \Delta\nabla f = \nabla\Delta f + \mathrm{Ric}(\nabla f, \cdot) \Longrightarrow \text{scalar Bochner identity.}$$

The corresponding Lean modules are:

1. `Bochner/Basic.lean`,
2. `Bochner/Bochner.lean`,
3. `Bochner/ComponentBridge.lean` and `Bochner/CoordinateBochner.lean`,
4. `Bochner/ClassicalCoordinateBochner.lean`,
5. `Bochner/ConcreteCoordinateBochner.lean`.

13.3 Bundle-valued connection algebra (`Bochner/Basic.lean`)

Definition 13.2 (Bundle-valued connection algebra)**.** *The abstract Bochner package introduces five groups:*

$$\Omega^1(\mathrm{End}\,E),\ \Omega^2(\mathrm{End}\,E),\ \Omega^0(E),\ \Omega^1(E),\ \Omega^2(E),$$

together with

$$d_{\mathrm{End}},\ d_{E,0},\ d_{E,1},\ \wedge,$$

and three compatible actions, denoted uniformly by "$\cdot$", in degrees 0, 1, and 2. The axioms say that $d_{E,1}$ is additive, that $d_{E,1}(d_{E,0}s) = 0$, that $d_{E,1}$ satisfies the Leibniz rule against the $\mathrm{End}(E)$-action, and that the iterated action is associative:

$$A \cdot (B \cdot s) = (A \wedge B) \cdot s.$$

For a connection form A the Lean file defines

$$\mathcal{F}(A) := d_{\mathrm{End}}A + A \wedge A,$$

$$\nabla s := d_{E,0}s + A \cdot s, \qquad \nabla\omega := d_{E,1}\omega + A \cdot \omega,$$

and

$$\mathcal{F}(A) \cdot s := \mathcal{F}(A)s.$$

13.4 Abstract curvature-action theorem (`Bochner/Bochner.lean`)

Theorem 13.3 (Abstract Bochner commutator)**.** *For every section s,*

$$\nabla(\nabla s) = \mathcal{F}(A) \cdot s.$$

Proof. Expand both covariant derivatives:

$$\nabla(\nabla s) = d_{E,1}(d_{E,0}s + A \cdot s) + A \cdot (d_{E,0}s + A \cdot s).$$

Use additivity, $d_{E,1}(d_{E,0}s) = 0$, and the Leibniz rule for $d_{E,1}(A \cdot s)$ to obtain

$$d_{\mathrm{End}}A \cdot s - A \cdot d_{E,0}s + A \cdot d_{E,0}s + (A \wedge A) \cdot s.$$

The middle terms cancel, leaving

$$(d_{\mathrm{End}}A + A \wedge A) \cdot s = \mathcal{F}(A) \cdot s.$$

Again the Lean proof is the same compact pattern as before: `simp`, two `rw` blocks, then `abel`. □

13.5 Component bridge and coordinate commutator (`Bochner/ComponentBridge.lean`, `Bochner/Coordi`

The coordinate package specializes the abstract theorem to covectors. It uses

$$C^1 = (\mathrm{Fin}\, n)^3 \to \mathbb{R}, \qquad C^2 = (\mathrm{Fin}\, n)^4 \to \mathbb{R}, \qquad \alpha : \mathrm{Fin}\, n \to \mathbb{R},$$

and defines the action of a connection on a covector by

$$(A \cdot \alpha)_{\mu k} = \sum_{\ell} A_{k\ell,\mu}\, \alpha_\ell.$$

The next action is

$$(A \cdot \omega)_{\mu\nu k} = \sum_{\ell} \big(A_{k\ell,\mu}\omega_{\nu\ell} - A_{k\ell,\nu}\omega_{\mu\ell}\big),$$

while curvature acts by

$$(F \cdot \alpha)_{\mu\nu k} = \sum_{\ell} F_{k\ell,\mu\nu}\alpha_\ell.$$

Theorem 13.4 (Coordinate covector commutator)**.** *For every covector field α,*

$$D_{\mu\nu}\alpha_k + \sum_{\ell} \big(A_{k\ell,\mu}(\nabla_\nu \alpha)_\ell - A_{k\ell,\nu}(\nabla_\mu \alpha)_\ell\big) = \sum_{\ell} R_{k\ell,\mu\nu}\, \alpha_\ell.$$

Proof. Apply the component bridge to Theorem 13.3 and use the projection extracting the (μ, ν, k)-entry of an E-valued 2-form. In Lean, `proj_covector_commutator_eq_explicit` identifies the left side, `proj_curvature_action_eq_explicit` identifies the right side, and `classical_covector_commutator` combines them. □

13.6 Trace step and scalar identity (`Bochner/ClassicalCoordinateBochner.lean`, `Bochner/ConcreteCoor`

The trace file defines:

$$(\nabla^*\nabla\, \alpha)_j = \sum_i T_{iij}, \qquad \beta_j = \sum_i T_{jii},$$

$$\mathrm{Ric}_{jl} = \sum_i R_{ilij}, \qquad (\mathrm{Ric} \cdot \alpha)_j = \sum_l \mathrm{Ric}_{jl}\ \alpha_l.$$

Theorem 13.5 (Trace commutator)**.** *If*

$$T_{ijk} - T_{jik} = \sum_l R_{klij}\, \alpha_l,$$

and $T_{ijk} = T_{ikj}$, then

$$(\nabla^*\nabla\, \alpha)_j = \beta_j + (\mathrm{Ric} \cdot \alpha)_j.$$

Proof. Use the symmetry $T_{iij} = T_{iji}$ to rewrite the rough Laplacian, then substitute the commutator relation with $k = i$:

$$\sum_i T_{iji} = \sum_i T_{jii} + \sum_i \sum_l R_{ilij}\, \alpha_l.$$

The double sum is exactly the Ricci action; the Lean proof packages that final identification as `ricciAction_eq_double_sum`. □

The concrete file then records the standard formulas

$$\langle u, v\rangle = \sum_i u_i v_i, \qquad \|H\|^2 = \sum_{i,j} H_{ij}^2,$$

and

$$\Delta|\alpha|^2 = 2\|H\|^2 + 2\langle \alpha, \nabla^*\nabla\, \alpha\rangle.$$

Taking the inner product of Theorem 13.5 with α and distributing the sums yields the scalar identity stated in Theorem 13.1. The Lean proof first proves the inner-product rewrite and then finishes with two short `ring` calculations.

14 Weitzenböck Identity

14.1 Main theorem

Main Theorem 14.1 (One-form Weitzenböck identity)**.** *Let D be coordinate connection data, let A be a connection 1-form, and let $S(\alpha)_{ijk}$ assign to each covector field α a second covariant derivative tensor. Assume the Bochner commutator bridge*

$$[\nabla_i, \nabla_j]\alpha_k = S(\alpha)_{ijk} - S(\alpha)_{jik}.$$

Then the formalized one-form Hodge Laplacian satisfies

$$(\Delta_H \alpha)_j = (\nabla^* \nabla \alpha)_j + \sum_l \mathrm{Ric}_{jl}\ \alpha_l.$$

14.2 Proof architecture

The Weitzenböck branch deliberately reuses the Bochner branch:

1. introduce an abstract one-form Hodge package;
2. prove $\Delta_H \alpha = \nabla^* \nabla \alpha + \mathcal{R}(\alpha)$ abstractly;
3. build the concrete $d\delta$, δd, and rough operators from a second-derivative family;
4. identify the curvature term by tracing the Bochner commutator.

14.3 Abstract one-form Hodge algebra (`Weitzenbock/Basic.lean`, `Weitzenbock/Weitzenbock.lean`)

Definition 14.1 (One-form Hodge algebra)**.** *A one-form Hodge algebra consists of an abelian group Ω^1 together with four endomorphisms*

$$d\delta, \qquad \delta d, \qquad \nabla^* \nabla, \qquad \mathcal{R},$$

such that

$$\delta d\, \alpha = \nabla^* \nabla\, \alpha + \mathcal{R}(\alpha) - d\delta\, \alpha$$

for every α.

The Hodge Laplacian is then defined by

$$\Delta_H \alpha := d\delta\, \alpha + \delta d\, \alpha.$$

Theorem 14.2 (Abstract Weitzenböck identity)**.** *For every one-form Hodge algebra and every one-form α,*

$$\Delta_H \alpha = \nabla^* \nabla\, \alpha + \mathcal{R}(\alpha).$$

Proof. Substitute the defining relation for $\delta d\, \alpha$:

$$\Delta_H \alpha = d\delta\, \alpha + \big(\nabla^* \nabla\, \alpha + \mathcal{R}(\alpha) - d\delta\, \alpha\big),$$

and cancel the two $d\delta$ terms. Lean proves this by one rewrite followed by `abel`. □

14.4 Coordinate package and Bochner trace bridge (`CoordinateWeitzenbock.lean`, `ClassicalCoordinat`

The coordinate layer first packages arbitrary operators

$$d\delta,\ \delta d,\ \nabla^* \nabla,\ \mathcal{R} : \Omega^1 \to \Omega^1$$

into the abstract Hodge algebra. The classical bridge then builds these operators from a second-derivative family S:

$$(d\delta\, \alpha)_j = -\sum_i S(\alpha)_{jii},$$

$$(\nabla^* \nabla\, \alpha)_j = -\sum_i S(\alpha)_{iij},$$

$$(\delta d\, \alpha)_j = \sum_i \big(-S(\alpha)_{iij} + S(\alpha)_{iji}\big).$$

The curvature operator is defined by the Ricci action from the Bochner branch:

$$(\mathcal{R}\alpha)_j = \sum_l \mathrm{Ric}_{jl}\ \alpha_l.$$

Theorem 14.3 (Trace of the Bochner commutator)**.** *Under the Bochner commutator bridge,*

$$\sum_i S(\alpha)_{iji} = \sum_i S(\alpha)_{jii} + (\mathcal{R}\alpha)_j.$$

Proof. Apply the Bochner commutator with the index choice $k = i$:

$$S(\alpha)_{iji} - S(\alpha)_{jii} = \sum_l R_{ilij}\, \alpha_l.$$

Summing over i yields the stated identity. The right-hand side is then recognized as the Ricci action by the same double-sum lemma used in the Bochner branch. □

Corollary 14.4 (Codifferential square decomposition)**.** *The operators built from S satisfy*

$$\delta d\, \alpha = \nabla^* \nabla\, \alpha + \mathcal{R}\alpha - d\delta\, \alpha.$$

Proof. Substitute Theorem 14.3 into the definition of δd:

$$(\delta d\, \alpha)_j = -\sum_i S(\alpha)_{iij} + \sum_i S(\alpha)_{iji} = -\sum_i S(\alpha)_{iij} + \sum_i S(\alpha)_{jii} + (\mathcal{R}\alpha)_j.$$

The first sum is $(\nabla^* \nabla\, \alpha)_j$, while the second is $-(d\delta\, \alpha)_j$. □

This corollary is exactly the field `deltaD_eq_rough_plus_curvature_minus_dDelta` in the Lean `HodgeData` structure, so Theorem 14.2 applies and produces the coordinate one-form Weitzenböck identity.

14.5 Concrete formula and pairing corollary (`ConcreteCoordinateWeitzenbock.lean`)

Theorem 14.5 (Concrete one-form formula)**.** *For every covector field α,*

$$(\Delta_H \alpha)_j = (\nabla^* \nabla \alpha)_j + \sum_l \mathrm{Ric}_{jl}\ \alpha_l.$$

Proof. This is the direct specialization of Theorem 14.2 under the coordinate package built in the previous subsection. The concrete Lean theorem simply unfolds the Ricci action as an explicit finite sum. □

Corollary 14.6 (Pairing form)**.** *For every covector field α,*

$$\langle \alpha, \Delta_H \alpha \rangle = \langle \alpha, \nabla^* \nabla\, \alpha \rangle + \langle \alpha, \mathcal{R}\alpha \rangle.$$

Proof. Multiply Theorem 14.5 by α_j, sum over j, and distribute the finite sum. This is exactly the calculation carried out in `concrete_weitzenbock_pairing`, with the final step closed by `ring` and `Finset.sum_add_distrib`. □

15 Comparison Geometry

15.1 Model space ODE solutions (`Comparison/ODEModels.lean`)

Definition 15.1 (Model comparison function)**.** *For $k \in \mathbb{R}$, the model function $\mathrm{sn}_k : \mathbb{R} \to \mathbb{R}$ is*

$$\mathrm{sn}_k(t) := \begin{cases} \dfrac{\sin(\sqrt{k}\,t)}{\sqrt{k}} & k > 0, \\ t & k = 0, \\ \dfrac{\sinh(\sqrt{-k}\,t)}{\sqrt{-k}} & k < 0. \end{cases}$$

This is the solution of $u'' + k\,u = 0$ with $u(0) = 0$, $u'(0) = 1$.

Definition 15.2 (Model-positive domain)**.** *The* model-positive domain *is*

$$\texttt{modelPosDomain}(k) := \begin{cases} \big(0,\ \pi/\sqrt{k}\big) & k > 0, \\ (0, \infty) & k \le 0. \end{cases}$$

Lemma 15.3 (`sn_pos_of_mem_modelPosDomain`)**.** $\mathrm{sn}_k(t) > 0$ *for all $t \in$ `modelPosDomain`(k).*

Proof. Split by sign of k.

- $k > 0$: write $\mathrm{sn}_k(t) = \sin(\sqrt{k}\,t)/\sqrt{k}$. The argument $\sqrt{k}\,t \in (0, \pi)$ by definition of `modelPosDomain`, so $\sin(\sqrt{k}\,t) > 0$ by `Real.sin_pos_of_pos_of_lt_pi`, and $\sqrt{k} > 0$.
- $k = 0$: $\mathrm{sn}_0(t) = t > 0$.
- $k < 0$: write $\mathrm{sn}_k(t) = \sinh(\sqrt{-k}\,t)/\sqrt{-k}$. The argument $\sqrt{-k}\,t > 0$, so $\sinh(\sqrt{-k}\,t) > 0$ by `Real.sinh_pos_iff`.

□

15.2 Scalar Sturm comparison theorem (`Comparison/SturmComparison.lean`)

Main Theorem 15.1 (Scalar comparison)**.** *Let $k \in \mathbb{R}$ and let $y : \mathbb{R} \to \mathbb{R}$ satisfy:*

1. *$y(0) = 0$ and $y'(0) = 1$;*
2. *y is C^2 on `modelPosDomain`(k);*
3. *$y'' + k\,y \le 0$ on `modelPosDomain`(k) (the* subsolution *condition).*

Then $y(t) \le \mathrm{sn}_k(t)$ for all $t \in$ `modelPosDomain`(k).

The proof proceeds through four lemmas, following the Wronskian route.

15.2.1 Wronskian and its derivative

Definition 15.4 (Scalar Wronskian)**.**

$$W(t) := \texttt{scalarWronskian}(k, y)(t) := y'(t)\,\mathrm{sn}_k(t) - y(t)\,\mathrm{sn}_k'(t).$$

Lemma 15.5 (`hasDerivAt_scalarWronskian`)**.** *At each $t \in$ `modelPosDomain`(k),*

$$W'(t) = \big(y''(t) + k\,y(t)\big) \cdot \mathrm{sn}_k(t).$$

Proof. Apply the Leibniz rule twice. Setting $P(t) = y'(t)\,\mathrm{sn}_k(t)$ and $Q(t) = y(t)\,\mathrm{sn}_k'(t)$:

$$P'(t) = y''(t)\,\mathrm{sn}_k(t) + y'(t)\,\mathrm{sn}_k'(t),$$

$$Q'(t) = y'(t)\,\mathrm{sn}_k'(t) + y(t)\,\mathrm{sn}_k''(t) = y'(t)\,\mathrm{sn}_k'(t) + y(t) \cdot (-k\,\mathrm{sn}_k(t)).$$

Hence $W'(t) = P'(t) - Q'(t) = y''(t)\,\mathrm{sn}_k(t) + k\,y(t)\,\mathrm{sn}_k(t)$. In Lean, `hmul1` and `hmul2` compute P' and Q' via `HasDerivAt.mul`; a `convert` with a `ring` closes the goal. □

15.2.2 Wronskian is antitone

Lemma 15.6 (`deriv_scalarWronskian_nonpos_of_subsolution`). *Under the subsolution hypothesis, $W'(t) \le 0$ for all $t \in$ `modelPosDomain`(k).*

Proof. By Lemma 15.5, $W'(t) = (y'' + ky)(t) \cdot \mathrm{sn}_k(t)$. The subsolution gives $(y'' + ky)(t) \le 0$, and $\mathrm{sn}_k(t) > 0$ by Lemma 15.3. Hence $W'(t) \le 0$ by `mul_nonpos_of_nonpos_of_nonneg`. □

Lemma 15.7 (`antitoneOn_scalarWronskian_of_subsolution`). *W is antitone on `modelPosDomain`(k).*

Proof. Continuity of W on `modelPosDomain`(k) follows from the derivative hypothesis and `HasDerivAt.continuousAt`. Differentiability follows similarly. Lemma 15.6 gives $W'(t) \le 0$ everywhere. The Lean proof applies `antitoneOn_of_deriv_nonpos` with: the convexity of `modelPosDomain`(k) (`modelPosDomain_convex`), continuity of W, differentiability on the interior, and nonpositivity of W'. The interior step uses `isOpen_modelPosDomain k |>.interior_eq`. □

15.2.3 Initial value of the Wronskian

Lemma 15.8 (`scalarWronskianUpperBound_of_subsolution`). *Under the subsolution hypothesis, $W(t) \le 0$ for all $t \in$ `modelPosDoma`*

Proof. Suppose for contradiction that $W(t_0) > 0$ for some $t_0 \in$ `modelPosDomain`(k). Since W is antitone (Lemma 15.7) and $W(t_0) > 0$, we have $W(s) \ge W(t_0) > 0$ for all $s \le t_0$ in `modelPosDomain`(k). In particular $W(s) > 0$ for all small $s > 0$.

Meanwhile, the comparison ratio $r(t) = y(t)/\mathrm{sn}_k(t)$ satisfies $r(t) \to 1$ as $t \to 0^+$ (Lemma `tendsto_div_sn_at_zero_of_initial`, proved by L'Hôpital via `tendsto_slope_zero_right`). If $W(s) > 0$ near 0, then $r'(s) = W(s)/\mathrm{sn}_k(s)^2 > 0$ near 0, so r is increasing near 0. But $r \to 1$, so $r(s) < 1$ for small $s > 0$. The Lean argument chooses a small δ with $\delta \le \min(\delta_r, \delta_{\mathrm{sn}}, t_0/2, W(t_0)/4)$, sets $s = \delta/4$ and $2s = \delta/2$, applies the mean value theorem (`exists_hasDerivAt_eq_slope`) on $[s, 2s]$, and derives a contradiction from the sign of the slope. □

15.2.4 Comparison ratio is antitone and bounded above by 1

Theorem 15.9 (Proof of the scalar comparison). *$y(t) \le \mathrm{sn}_k(t)$ for all $t \in$ `modelPosDomain`(k).*

Proof. The ratio $r(t) = y(t)/\mathrm{sn}_k(t)$ satisfies

$$r'(t) = \frac{W(t)}{\mathrm{sn}_k(t)^2} \le 0$$

by Lemma 15.8 and $\mathrm{sn}_k(t)^2 > 0$. Hence r is antitone on `modelPosDomain`(k) (`antitoneOn_scalarComparisonRatio`: same `antitoneOn_of_deriv_nonpos` argument with the derivative formula from `hasDerivAt_scalarComparisonRatio`). Since $r(t) \to 1$ as $t \to 0^+$ (Lemma 15.8), and r is antitone, $r(t) \le 1$, i.e., $y(t) \le \mathrm{sn}_k(t)$. □

15.3 Scalar Jacobi comparison bridge (`Comparison/ScalarJacobiComparison.lean`)

Definition 15.10 (`HasScalarJacobiComparison`). *Given a Jacobi field J along a geodesic and a model function sn_k, the predicate `HasScalarJacobiComparison` asserts that $\|J(t)\| \le c\,\mathrm{sn}_k(t)$ for some constant c and all t in the domain.*

Theorem 15.11 (`hasScalarJacobiComparison_of_subsolution`). *If the Jacobi norm $y(t) = \|J(t)\|$ satisfies the subsolution inequality $y'' + k\,y \le 0$ with $y(0) = 0$, $y'(0) = 1$, then `HasScalarJacobiComparison` holds.*

Proof. Apply the scalar comparison theorem (Theorem 15.9) to y: this gives $y(t) \le \mathrm{sn}_k(t)$, which with $c = 1$ is exactly `HasScalarJacobiComparison`. □

15.4 Rauch norm core (`Comparison/RauchNormCore.lean`)

This subsection develops the owner-local infrastructure for Rauch comparison, working with the squared norm $u(t) = \|J(t)\|^2$ of a coordinate Jacobi field to avoid the singular denominator that arises at zeros of J.

Definition 15.12 (Squared norm and inner product). *For $v : \mathrm{Fin}\, n \to \mathbb{R}$:*

$$\mathit{vecNormSq}(v) := \sum_i v_i^2, \qquad \mathit{vecInner}(v, w) := \sum_i v_i w_i, \qquad \mathit{matrixQuadForm}(A, v) := \langle Av, v\rangle = \sum_{i,j} A_{ij} v_j v_i.$$

The squared Jacobi norm is $u(t) :=$ `jacobi_normSq`$(J)(t) =$ `vecNormSq`$(J(t))$.

Theorem 15.13 (`hasDerivAt_jacobi_normSq`). *If each component J_i has derivative $J_i'(t)$ at t, then*

$$\frac{d}{dt}\, u(t) = 2\, \mathit{vecInner}(J(t),\, J'(t)) = 2 \sum_i J_i(t)\, J_i'(t).$$

Proof. `HasDerivAt.fun_sum` differentiates the sum $\sum_i J_i(s)^2$ termwise via the chain rule: each term gives $\frac{d}{ds} J_i(s)^2 = 2J_i(t)J_i'(t)$. Summing and rewriting via `Finset.mul_sum` gives the stated formula. □

Definition 15.14 (`HasRayleighUpperBound`). *`HasRayleighUpperBound`(A, k) asserts that for all t and all $v : \mathrm{Fin}\, n \to \mathbb{R}$,*

$$\langle A(t)\, v,\, v\rangle \le k\, \|v\|^2.$$

This encodes the sectional curvature upper bound $\sec \le k$ *in coordinate form.*

Theorem 15.15 (`jacobi_normSq_secondDeriv_lower_bound`). *Under the Rayleigh upper bound $\langle A(t)\xi, \xi\rangle \le k\|\xi\|^2$, the second derivative of u satisfies*

$$u''(t) \ge 2\big(\|J'(t)\|^2 - k\, \|J(t)\|^2\big).$$

Proof. From the Jacobi equation $J'' = -A(t)J$, the second derivative of $u = \sum_i J_i^2$ is

$$u''(t) = 2\sum_i \big[(J_i')^2 + J_i J_i''\big] = 2\big[\|J'\|^2 - \langle A(t)J, J\rangle\big].$$

Applying the Rayleigh bound $\langle A(t)J, J\rangle \le k\|J\|^2$ and using `nlinarith` gives $u''(t) \ge 2(\|J'\|^2 - k\|J\|^2)$. □

Corollary 15.16 (`jacobi_normSq_convex_of_nonpositive_curvature`). *If $k \le 0$, then $u''(t) \ge 0$: the squared norm of any Jacobi field is convex.*

Proof. For $k \le 0$: both $\|J'\|^2 \ge 0$ and $-k\|J\|^2 \ge 0$, so $u'' \ge 2(\|J'\|^2 - k\|J\|^2) \ge 0$. The Lean proof combines `vecNormSq_nonneg`, the Rayleigh bound, and `nlinarith`. □

15.5 Scalar convexity for Cartan–Hadamard (`Comparison/ScalarConvexity.lean`)

This subsection proves the 1D analytical core of the Cartan–Hadamard theorem.

Main Theorem 15.2 (`positive_of_nonneg_second_deriv`). *Let $y : \mathbb{R} \to \mathbb{R}$ be differentiable on $(0, \infty)$ with $y(0) = 0$, $y'(0) > 0$, and $y''(t) \ge 0$ for all $t > 0$. Then $y(t) > 0$ for all $t > 0$.*

Proof. The proof applies the mean value theorem twice.

Step 1: y' is nondecreasing. (`deriv_nondecreasing`) For any $s > 0$, apply the MVT to y' on $[0, s]$: there exists $c \in (0, s)$ with $y'(s) - y'(0) = y''(c) \cdot s$. Since $y''(c) \ge 0$ and $s > 0$, we get $y'(s) \ge y'(0) > 0$. In Lean: `exists_hasDerivAt_eq_slope` gives the mean value c; the product $y''(c) \cdot s \ge 0$ is handled by `nlinarith`.

Step 2: $y(t) > 0$. Apply the MVT to y on $[0, t]$: there exists $c \in (0, t)$ with $y(t) - y(0) = y'(c) \cdot t$. Since $y(0) = 0$ and $y'(c) \ge y'(0) > 0$ by Step 1, we get $y(t) = y'(c) \cdot t > 0$. □

Remark 15.17. Under nonpositive sectional curvature, the Jacobi field norm $\|J(t)\|$ satisfies the hypotheses of Theorem 15.2 (with $y = \|J\|$, $y'(0) = \|J'(0)\| > 0$, $y'' \ge 0$ by Corollary 15.16). This immediately gives $\|J(t)\| > 0$ for $t > 0$, i.e., the geodesic has no conjugate points.

15.6 Bishop–Gromov monotonicity (`Comparison/BishopGromovMonotonicity.lean`)

Theorem 15.18 (`bishopGromov_monotonicity_of_local_inequality`). *Given a `VolumeGrowthFromDensityData` structure with a `HasLogJacobianDifferentialInequality` hypothesis, the Bishop–Gromov volume ratio* $\mathrm{vol}(B_r(p))/V_k(r)$ *is monotone nonincreasing in r.*

Proof. This is a thin wrapper: it invokes `bishopGromov_monotonicity_of_density_growth` from `Comparison/VolumeGrowthFromDens` which establishes the 1D integral argument — if the log-Jacobian defect $(\log J - \log J_k)' \le 0$ along each radial ray, then the ratio integrand is nonincreasing, and hence so is the ratio itself. □

16 Hopf–Rinow: Metric Foundations

Remark 16.1 (Proof status). Four files in this module are fully proved with no external input: `Properness`, `DistanceRealizer`, `LengthCompactness`, and `LocalCompactness`. The remaining files (`CompleteProper`, `MetricLength`, `MinimizerExistence`, `SmoothGeo` are partial scaffolds with pending owner theorems and are not covered here.

16.1 Properness and completeness (`HopfRinow/Properness.lean`)

Definition 16.2 (Riemannian properness and completeness). *For a pseudo-metric space X:*

- *`RiemannianProper`(X) :⇔ `ProperSpace`(X) (every closed bounded set is compact).*
- *`RiemannianComplete`(X) :⇔ `CompleteSpace`(X) (every Cauchy sequence converges).*

These are package-level wrappers around the Mathlib typeclasses, introduced to give a stable interface independent of typeclass inference paths.

Theorem 16.3 (`riemannianComplete_of_proper`). *If X is a proper metric space, then X is complete: `RiemannianProper`(X) implies `RiemannianComplete`(X).*

Proof. `ProperSpace` implies `CompleteSpace` in Mathlib via `infer_instance`: a proper space is sequentially compact on closed balls, which implies Cauchy sequences converge. The Lean proof unfolds the definitions and closes by typeclass synthesis. □

Theorem 16.4 (`riemannianProper_of_properSpace`). *If the `ProperSpace` typeclass is in scope, then `RiemannianProper`(X) holds by definition unfolding.*

16.2 Distance realizers and minimizing geodesics (`HopfRinow/DistanceRealizer.lean`)

Definition 16.5 (Constant-speed geodesic). *`IsGeodesicOnWithSpeed`(X, γ, c, I) asserts that $\gamma : \mathbb{R} \to X$ is a constant-speed curve on $I \subseteq \mathbb{R}$ with speed $c \geq 0$:*

$$\forall s, t \in I, \quad \operatorname{dist}(\gamma(s), \gamma(t)) = c \cdot |t - s|.$$

`IsMinimizingGeodesicBetween`(X, p, q, γ) asserts that γ connects p to q on $[0, 1]$ and realizes the distance: $\gamma(0) = p$, $\gamma(1) = q$, `IsGeodesicOnWithSpeed`$(\gamma, \operatorname{dist}(p, q), [0, 1])$.

`IsDistanceRealizerBetween`(X, p, q, γ) stores at each $t \in [0, 1]$:

$$\operatorname{dist}(p, \gamma(t)) = t \cdot \operatorname{dist}(p, q), \qquad \operatorname{dist}(\gamma(t), q) = (1 - t) \cdot \operatorname{dist}(p, q).$$

Theorem 16.6 (`IsGeodesicOnWithSpeed.restrict_Icc`). *A constant-speed geodesic on $[a, b]$ restricts to a constant-speed geodesic on any subinterval $[a', b'] \subseteq [a, b]$.*

Proof. The distance formula $\operatorname{dist}(\gamma(s), \gamma(t)) = c|t - s|$ holds for all $s, t \in [a, b]$; restricting to $[a', b'] \subseteq [a, b]$ preserves it. The Lean proof applies `IsGeodesicOnWithSpeed.mono` with the subset inclusion. □

Theorem 16.7 (`IsMinimizingGeodesicBetween.toIsDistanceRealizerBetween`). *Every minimizing geodesic is a distance realizer.*

Proof. For $t \in [0, 1]$, the constant-speed property gives $\operatorname{dist}(\gamma(t), \gamma(0)) = \operatorname{dist}(p, q) \cdot t$, i.e., $\operatorname{dist}(p, \gamma(t)) = t \cdot \operatorname{dist}(p, q)$. Similarly, $\operatorname{dist}(\gamma(t), q) = (1 - t) \cdot \operatorname{dist}(p, q)$. In Lean, both directions are `calc` blocks that manipulate $|t - 0|$ and $|t - 1|$ via `abs_of_nonneg` / `abs_of_nonpos`, then simplify with `mul_comm` and the constant-speed formula. □

Theorem 16.8 (`IsDistanceRealizerBetween.toIsMinimizingGeodesicBetween_of_lipschitz`). *A distance realizer that is additionally Lipschitz with constant $\operatorname{dist}(p, q)$ is a minimizing geodesic.*

Proof. From the distance-realizer lower bound `lowerDistBound`:

$$(t - s) \operatorname{dist}(p, q) \leq \operatorname{dist}(\gamma(s), \gamma(t)),$$

and the Lipschitz upper bound $\operatorname{dist}(\gamma(s), \gamma(t)) \leq \operatorname{dist}(p, q) \cdot |t - s|$, the two sides agree: $\operatorname{dist}(\gamma(s), \gamma(t)) = \operatorname{dist}(p, q) \cdot |t - s|$. This gives `IsGeodesicOnWithSpeed`, and together with the endpoint conditions from the realizer this assembles into `IsMinimizingGeodesicBetwe`

The lower bound itself is proved by the triangle inequality: $\operatorname{dist}(p, q) \leq \operatorname{dist}(p, \gamma(s)) + \operatorname{dist}(\gamma(s), \gamma(t)) + \operatorname{dist}(\gamma(t), q)$, substituting the realizer formulas and solving via `nlinarith`. □

Corollary 16.9. *The minimizing-geodesic and distance-realizer predicates are equivalent given the Lipschitz bound:*

`IsMinimizingGeodesicBetween`$(p, q, \gamma) \iff$ `IsDistanceRealizerBetween`$(p, q, \gamma) \wedge$ `IsGeodesicOnWithSpeed`$(\gamma, \operatorname{dist}(p, q), [0, 1])$.

16.3 Length-compactness via Arzelà–Ascoli (`HopfRinow/LengthCompactness.lean`)

Definition 16.10 (Length-compact container)**.** *For a metric space X, base point x_0, and radii $R, L \geq 0$:*

$$\texttt{lengthCompactContainer}(x_0, R, L) := \left\{\gamma \in C([0,1], X) \;\middle|\; \operatorname{dist}(\gamma(t), x_0) \leq R \;\forall t,\; \texttt{LipschitzWith}(L, \gamma)\right\}.$$

Main Theorem 16.1 (`isCompact_lengthCompactContainer`)**.** *If X is a proper metric space (`RiemannianProper`), then `lengthCompa` is compact for every x_0, R, L.*

Proof. The proof lifts the family to bounded continuous functions $[0,1] \to^b X$ via `BoundedContinuousFunction.mkOfCompact` and applies Mathlib's Arzelà–Ascoli theorem (`BoundedContinuousFunction.arzela_ascoli₂`). The three hypotheses of Arzelà–Ascoli are verified:

1. **Target compactness:** $\overline{B}(x_0, R)$ is compact because X is proper.
2. **Pointwise containment:** Every γ in the container maps into $\overline{B}(x_0, R)$ by definition.
3. **Equicontinuity:** The uniform Lipschitz bound L gives `LipschitzWith.uniformEquicontinuous` (every γ in the container has the same Lipschitz constant L).

The closed-ness of the range-bounded set and the Lipschitz set in the function space are proved separately (`isClosed_boundedRange`, `isClosed_uniformLipschitz`) to show the container is a closed subset of the Arzelà–Ascoli compact. The image under the continuous map `toContinuousMap` : $(C([0,1] \to^b X)) \to C([0,1], X)$ then gives compactness of the original container. □

Theorem 16.11 (`hasLengthCompactness_of_proper`)**.** *If X is proper, then every uniformly Lipschitz bounded family of curves on $[0,1]$ has a compact superset.*

Proof. Given a family $S \subseteq$ `lengthCompactContainer`(x_0, R, L), Theorem 16.1 gives a compact set $K =$ `lengthCompactContainer`$(x_0, L$
S. □

16.4 Local compactness from normal neighborhoods (`HopfRinow/LocalCompactness.lean`)

Theorem 16.12 (`exists_compact_small_closedBall`)**.** *Let Γ be a smooth Christoffel field and $p \in \mathbb{R}^n$. There exists $r > 0$ such that the closed ball $\overline{B}(p, r)$ is compact and contained in the normal neighborhood `normalNeighborhood`(Γ, p).*

Proof. The normal neighborhood `normalNeighborhood`(Γ, p) is open (Theorem 8.3) and contains p (Theorem 8.4). By `Metric.isOpen_i` there exists $r > 0$ with $B(p, r) \subseteq$ `normalNeighborhood`(Γ, p). Take $r' = r/2$: the closed ball $\overline{B}(p, r')$ is compact because it is a closed bounded subset of $\mathbb{R}^n$ (Heine–Borel via `isCompact_closedBall`), and $\overline{B}(p, r') \subseteq B(p, r) \subseteq$ `normalNeighborhood`(Γ, p) since every point of $\overline{B}(p, r')$ is at distance $\leq r/2 < r$ from p. □

17 Summary of the Consolidated Document

Metric

Source file	Key result	Section
`Metric/Basic.lean`	Inner-product identities, positivity, and Cauchy–Schwarz	§1
`Metric/CoordinateMetric.lean`	Coordinate metric tensor, inverse pair, and extensionality	§1
`Metric/Musical.lean`	Flat/sharp musical maps and injectivity	§1
`Metric/SmoothMetric.lean`	Smooth metric field package	§1

Levi-Civita

Source file	Key result	Section
`LeviCivita/Basic.lean`	Abstract Levi-Civita axioms and uniqueness framework	§2
`LeviCivita/Koszul.lean`	Koszul formula and Levi-Civita characterization	§2
`LeviCivita/LeviCivita.lean`	Canonical Levi-Civita connection and uniqueness	§2
`LeviCivita/CoordinateFields.lean`	Smooth coordinate field and Christoffel packages	§2
`LeviCivita/LocalExistence.lean`	Local coordinate Levi-Civita construction	§2
`LeviCivita/CoordinateChristoffel.lean`	Christoffel-to-covariant-derivative bridge	§2
`LeviCivita/Globalization.lean`	Global existence and uniqueness wrappers	§2

ODE Infrastructure

Source file	Key result	Section
`ODE/Core.lean`	Gronwall flow estimates and fixed-time Lipschitz bound	§3
`ODE/UniformRemainder.lean`	Compact-tube uniform Taylor remainder estimate	§3
`ODE/Linearized.lean`	Short-time linearised solution operator (CLM)	§3
`ODE/LinearizedGlobal.lean`	Global linearised CLM via piecewise assembly	§3
`ODE/FlowFDeriv.lean`	Fixed-time Fréchet differentiability of C^1 flows	§3
`ODE/FlowContDiff.lean`	C^1 regularity of the flow in the initial condition	§3

Parallel Transport

Source file	Key result	Section
`ParallelTransport/Basic.lean`	Fields along curves and transport operators	§4
`ParallelTransport/Isometry.lean`	Parallel transport preserves the metric pairing	§4
`ParallelTransport/Frames.lean`	Transported orthonormal frames	§4

Curvature

Source file	Key result	Section
`Curvature/Basic.lean`	Curvature operator and tensor packaging	§5
`Curvature/Symmetries.lean`	First-pair skew symmetry and first Bianchi identity	§5
`Curvature/SectionalRicci.lean`	Sectional and Ricci curvature definitions	§5

Geodesic ODE

Source file	Key result	Section
`Geodesic/Basic.lean`	Phase-space state and initial-condition package	§6
`Geodesic/Spray.lean`	Geodesic spray and scaling ODE lemmas	§6
`Geodesic/CoordinateEquation.lean`	Acceleration $\ddot{x}^i = -\Gamma^i{}_{jk}\dot{x}^j\dot{x}^k$	§6
`Geodesic/LocalExistence.lean`	Picard–Lindelöf local geodesic existence	§6
`Geodesic/LocalUniqueness.lean`	Uniqueness from matching initial data	§6
`Geodesic/FlowDifferentiability.lean`	Fréchet differentiability of the geodesic flow	§6

Exponential Map

Source file	Key result	Section
`Exponential/Basic.lean`	$\exp_p$ definition and $\exp_p(0) = p$	§7
`Exponential/DifferentialAtZero.lean`	$D\exp_p(0) = \mathrm{id}$	§7
`Exponential/Differentiability.lean`	Fréchet differentiability on the chart source	§7
`Exponential/DexpContinuity.lean`	C^1 regularity and radial continuity of $d\exp$	§7
`Exponential/LocalInverse.lean`	Local homeomorphism via the Inverse Function Theorem	§7
`Exponential/LocalRiemannianData.lean`	Smooth metric + inverse + pos. def. data package	§7
`Exponential/NormalCoordinates.lean`	Normal coordinate chart and round-trip identities	§7
`Exponential/RadialBridge.lean`	Star-shaped radial source domain	§7
`Exponential/JacobiDexp.lean`	Directional derivative of exp (witness-free)	§7
`Exponential/RadialVariationIdentities.lean`	Owner-local radial variation derivative helpers	§7

Normal Neighborhoods

Source file	Key result	Section
`Minimization/NormalNeighborhoods.lean`	Openness, base-point membership, chart round-trips	§8

Riemannian Volume and Polar Coordinates

Source file	Key result	Section
`Measure/RiemannianVolume.lean`	Riemannian volume density $\sqrt{\det g}$	§9
`Measure/PolarCoordinates.lean`	Polar decomposition via the exponential map	§9
`Measure/PullbackDensity.lean`	Jacobian-weighted pullback measure	§9
`Measure/ExpJacobian.lean`	Hadamard determinant bound for $\det D\exp$	§9
`Measure/HadamardBound.lean`	$\lvert\det A\rvert \le \prod_i \lVert Ae_i\rVert$ on Euclidean maps	§9
`Measure/NormComparison.lean`	ℓ^∞–ℓ^2 norm comparison on $\mathrm{Fin}\, n \to \mathbb{R}$	§9

Variation

Source file	Key result	Section
`Variation/Basic.lean`	Abstract variation algebra	§10
`Variation/Variation.lean`	$\dot{\mathcal{F}} = d_\nabla \dot{A}$	§10
`Variation/ComponentBridge.lean`	Componentwise curvature variation	§10
`Variation/CoordinateVariation.lean`	Coordinate curvature variation formula	§10
`Variation/CurveEnergy.lean`	Speed-squared integrand $\lVert\dot\gamma\rVert^2$	§10
`Variation/CurveLength.lean`	Arc-length speed and unit-speed predicate	§10
`Variation/CurveVariation.lean`	Two-parameter family and field-along-variation	§10
`Variation/FirstVariationEnergy.lean`	First variation of energy; geodesic $\Rightarrow$ critical	§10
`Variation/FirstVariationLength.lean`	First variation of arc length; critical $\Leftrightarrow$ geodesic	§10
`Variation/IndexForm.lean`	Index form $I(J, J')$; bilinearity and symmetry	§10
`Variation/LeviCivitaVariationBridge.lean`	Levi-Civita bridge for $\dot\Gamma$	§10
`Variation/RicciScalarVariation.lean`	Ricci and scalar contraction formulas	§10
`Variation/SecondVariation.lean`	Second variation = index form	§10

Jacobi Fields

Source file	Key result	Section
`Jacobi/Basic.lean`	Jacobi field predicate and initial-data package	§11
`Jacobi/ExistenceUniqueness.lean`	Existence and uniqueness for Jacobi fields	§11
`Jacobi/VariationBridge.lean`	Geodesic-variation to Jacobi-field bridge	§11
`Jacobi/ConjugatePoints.lean`	Conjugate-point endpoint-value package	§11

Second Bianchi Identity

Source file	Key result	Section
`SecondBianchi/Basic.lean`	Abstract connection algebra for $d_\nabla \mathcal{F} = 0$	§12
`SecondBianchi/SecondBianchi.lean`	Abstract second Bianchi identity	§12
`SecondBianchi/ComponentBridge.lean`	Componentwise vanishing of $d_\nabla \mathcal{F}$	§12
`SecondBianchi/CoordinateBianchi.lean`	Coordinate component formula	§12
`SecondBianchi/ClassicalCoordinateBianchi.lean`	Cyclic covariant-derivative bridge	§12
`SecondBianchi/ConcreteCoordinateBianchi.lean`	Explicit d_1, d_2 specialization	§12
`SecondBianchi/SmoothCoordinateScaffold.lean`	Schwarz + Leibniz analytic layer	§12
`SecondBianchi/SmoothCoordinateBianchi.lean`	Smooth torsion-free theorem	§12

Bochner Identity

Source file	Key result	Section
`Bochner/Basic.lean`	Bundle-valued connection algebra	§13
`Bochner/Bochner.lean`	$\nabla^2 s = \mathcal{F} \cdot s$	§13
`Bochner/ComponentBridge.lean`	Abstract component bridge	§13
`Bochner/CoordinateBochner.lean`	Covector commutator formula	§13
`Bochner/ClassicalCoordinateBochner.lean`	Trace-to-Ricci step	§13
`Bochner/ConcreteCoordinateBochner.lean`	Scalar Bochner identity	§13

Weitzenböck Identity

Source file	Key result	Section
`Weitzenbock/Basic.lean`	Abstract one-form Hodge algebra	§14
`Weitzenbock/Weitzenbock.lean`	Abstract Weitzenböck identity	§14
`Weitzenbock/ComponentBridge.lean`	Component bridge for Δ_H	§14
`Weitzenbock/CoordinateWeitzenbock.lean`	Coordinate Hodge package	§14
`Weitzenbock/ClassicalCoordinateWeitzenbock.lean`	Bochner-based trace bridge	§14
`Weitzenbock/ConcreteCoordinateWeitzenbock.lean`	Concrete one-form and pairing formulas	§14

Comparison Geometry (fully proved files)

Source file	Key result	Section
`Comparison/ODEModels.lean`	Model function sn_k and positivity on model domain	§15
`Comparison/SturmComparison.lean`	Scalar Sturm comparison: $y \le \mathrm{sn}_k$	§15
`Comparison/ScalarJacobiComparison.lean`	Scalar Jacobi comparison bridge	§15
`Comparison/RauchNormCore.lean`	Squared Jacobi norm, Rayleigh bound, convexity for $k \le 0$	§15
`Comparison/ScalarConvexity.lean`	$y(0)=0$, $y'(0)>0$, $y'' \ge 0 \Rightarrow y(t) > 0$	§15
`Comparison/BishopGromovMonotonicity.lean`	Volume ratio monotonicity from log-Jacobian inequality	§15

Hopf–Rinow: Metric Foundations (fully proved files)

Source file	Key result	Section
`HopfRinow/Properness.lean`	`RiemannianProper`/`Complete` and proper $\Rightarrow$ complete	§16
`HopfRinow/DistanceRealizer.lean`	Minimizing geodesic $\Leftrightarrow$ distance realizer (+ Lipschitz)	§16
`HopfRinow/LengthCompactness.lean`	Arzelà–Ascoli compactness of Lipschitz curve families	§16
`HopfRinow/LocalCompactness.lean`	Compact closed balls inside normal neighborhoods	§16

The fully proved modules collected in this document span the complete infrastructure chain of Riemannian geometry, from metric algebra through global comparison theorems. They split into three layers.

The *analytic foundation* (ODE, Geodesic, Exponential, Normal Neighborhoods, Measure) builds the machinery of geodesic flows, the exponential map, and Riemannian volume, all with genuine proofs reaching into Mathlib's ODE and measure-theory libraries.

The *geometric infrastructure* (Metric, Levi-Civita, Parallel Transport, Curvature, Variation, Jacobi) assembles the classical Riemannian toolkit, with each layer proving both an abstract typeclass version and a concrete coordinate $\mathrm{Fin}\, n$ version.

The *curvature and comparison layer* (Second Bianchi, Bochner, Weitzenböck, Comparison, Hopf–Rinow foundations) closes the algebraic identities completely and provides the metric and analytical underpinning of the global comparison program. The two remaining `sorry`s (`Exponential/RadialGaussLemma` and `Minimization/MetricBridge`) are the sole boundary between this document and a fully proved Hopf–Rinow theorem.